

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

The raw UCI HAR inertial signals were first organized into a three-dimensional tensor so that the temporal structure required by recurrent networks was retained. Each activity sample consisted of 128 consecutive time steps and nine sensor channels.

The nine channels used were:

1. Body acceleration in the x direction
2. Body acceleration in the y direction
3. Body acceleration in the z direction
4. Body gyroscope in the x direction
5. Body gyroscope in the y direction
6. Body gyroscope in the z direction
7. Total acceleration in the x direction
8. Total acceleration in the y direction
9. Total acceleration in the z direction

Thus, one activity sample was represented by a matrix of size:

$$128 \times 9.$$

The selected dataset contained 1800 balanced samples, with 300 samples belonging to each of the six activity classes. These samples were divided into training, validation and internal test sets using a 70:15:15 ratio.

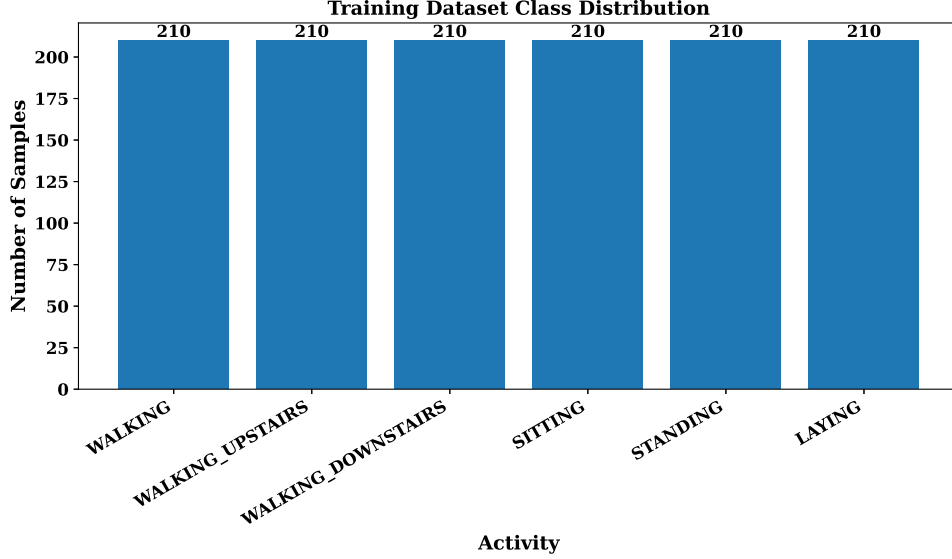


Figure 1: Class distribution of the selected balanced UCI HAR dataset.

The resulting training set contained 1260 samples, while the validation and internal test sets contained 270 samples each. The official UCI test set containing 2947 samples was kept separate and was used for the final evaluation of the trained models.

Training-Only Normalization

Normalization was performed independently for each of the nine sensor channels. The mean and standard deviation were calculated only from the training set:

$$\mu_f = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T X_{i,t,f}$$

and

$$\sigma_f = \sqrt{\frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T (X_{i,t,f} - \mu_f)^2}.$$

Each input value was then standardized using:

$$X'_{i,t,f} = \frac{X_{i,t,f} - \mu_f}{\sigma_f + \epsilon},$$

where ϵ is a small value used to avoid division by zero.

The calculated channel-wise normalization statistics were stored and reused for the validation and test data. Therefore, no statistics from the validation or test sets were used during training.

The preprocessing pipeline can be summarized as:

Raw Sensor Signals → 9 Channels → 128 Time Steps → Balanced Dataset → 70/15/15 Split → Training-only Normalization

After preprocessing, the input supplied to the recurrent models had the shape:

$$(N, 128, 9).$$

Here, N is the number of samples, 128 is the sequence length and 9 is the number of features at every time step.

6. Temporal Data Visualization

Before training the recurrent models, the temporal behaviour of the sensor signals was examined for different activity classes. Three representative activities were visualized: WALKING, SITTING and LAYING.

Each plot shows the variation of selected sensor channels over the 128 time steps of one activity sequence.

Temporal Sensor Patterns – WALKING

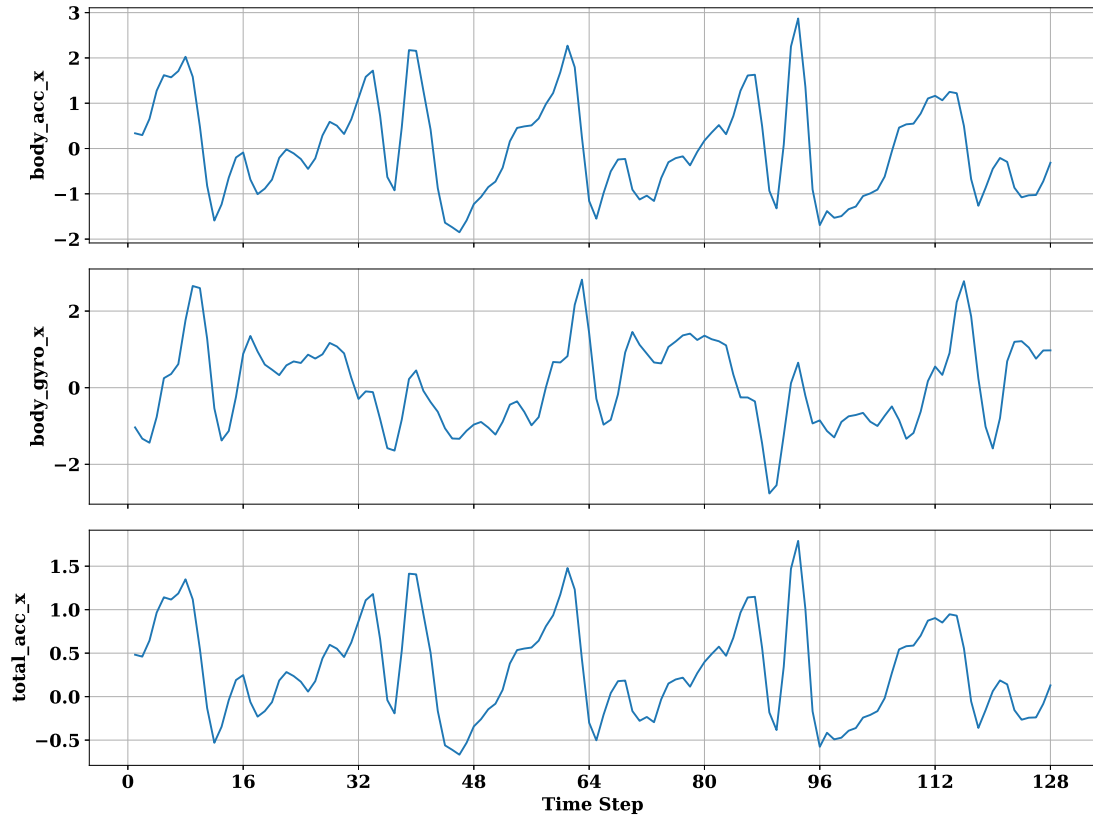


Figure 2: Temporal sensor signals for a WALKING sequence.

The WALKING sequence shows repeated fluctuations in the sensor signals. The body acceleration, body gyroscope and total acceleration contain noticeable peaks and troughs across the time steps. This indicates that the sensor values change continuously with the motion of the subject.

Temporal Sensor Patterns — SITTING

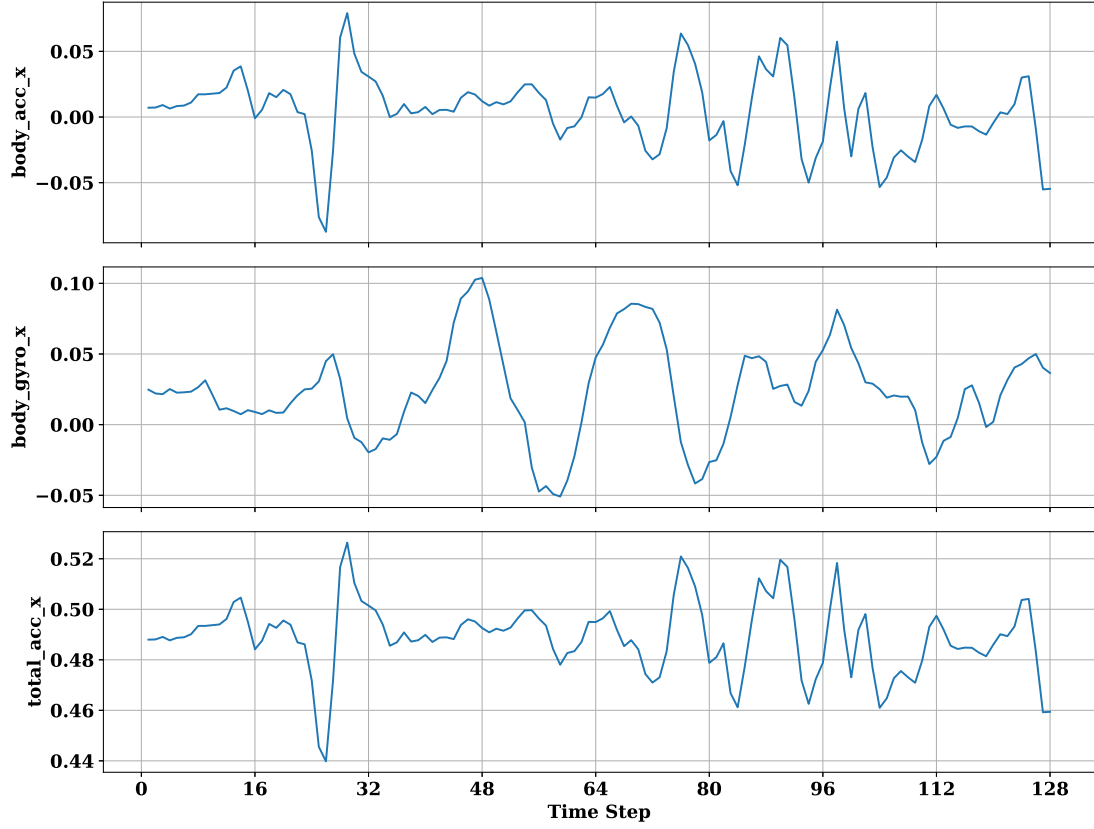


Figure 3: Temporal sensor signals for a SITTING sequence.

For SITTING, the variation in body acceleration is smaller compared with the walking sequence. The gyroscope signals show local variations, but the overall temporal pattern is less repetitive than that observed for walking.

Temporal Sensor Patterns — LAYING

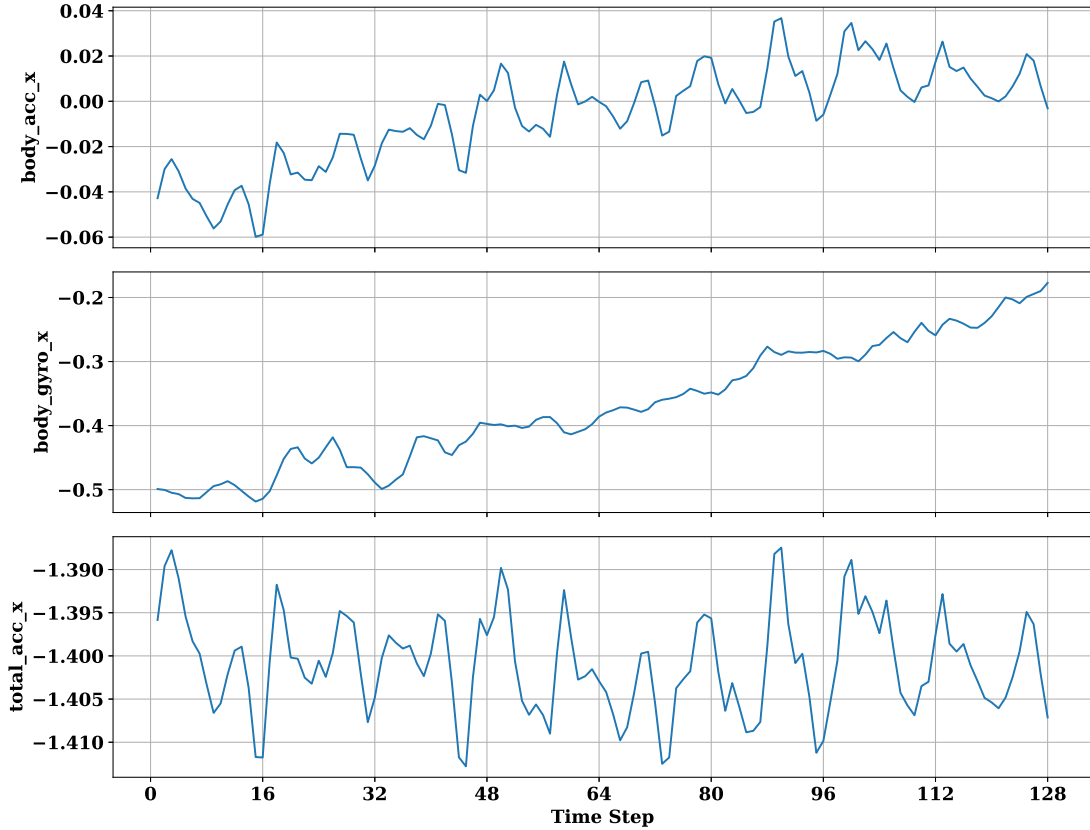


Figure 4: Temporal sensor signals for a LAYING sequence.

The LAYING sequence shows a different temporal pattern from the dynamic walking activity. The body acceleration and gyroscope signals vary more smoothly, while the total acceleration remains around a relatively stable region with smaller oscillations.

These plots show why sequence information is useful for human activity recognition. The classification problem is not based only on individual sensor values. The order in which the values change over the 128 time steps also contains information about the activity being performed. This temporal dependency is the main reason for using recurrent neural networks in this experiment.

7. Vanilla RNN Architecture

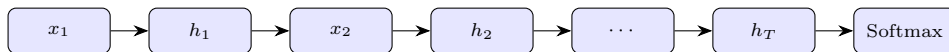
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

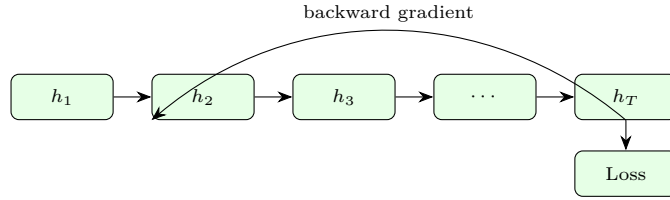
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

A simple recurrent computation was performed to understand how the hidden state is updated over successive time steps.

The following values were used:

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

The recurrent equation is:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

For the first time step:

$$h_1 = \tanh(0.5(0.5) + 0.8(0) + 0.1)$$

$$h_1 = \tanh(0.35)$$

$$h_1 = 0.336376$$

For the second time step:

$$h_2 = \tanh(0.5(0.7) + 0.8h_1 + 0.1)$$

$$h_2 = 0.616352$$

For the third time step:

$$h_3 = \tanh(0.5(0.2) + 0.8h_2 + 0.1)$$

$$h_3 = 0.599958$$

For the third time step:

$$h_3 = \tanh(0.5(0.2) + 0.8(0.616352) + 0.1)$$

$$h_3 = 0.599958$$

Therefore, the hidden-state sequence obtained from the numerical exercise was:

$$h_1 = 0.336376, \quad h_2 = 0.616352, \quad h_3 = 0.599958$$

The calculation shows that the hidden state at each time step depends on both the current input and the previous hidden state. Hence, information from earlier time steps is carried forward through the recurrent connection.

9. RNN Implementation

The Vanilla RNN was trained using the balanced HAR training set and the validation set described in the preprocessing section. The same preprocessing and classification setting was maintained so that the recurrent models could be compared under the same experimental conditions.

The training configuration was:

Parameter	Setting
Recurrent layer	SimpleRNN
Hidden units	32
Dropout	0.20
Dense layer	16 neurons, ReLU
Output layer	6 neurons, Softmax
Optimizer	Adam
Learning rate	0.001
Batch size	32
Maximum epochs	30

The model contained 1,974 trainable parameters. Training was performed for the configured maximum number of epochs, with validation performance monitored during training.

The training and validation loss curves are shown in Figure 5.

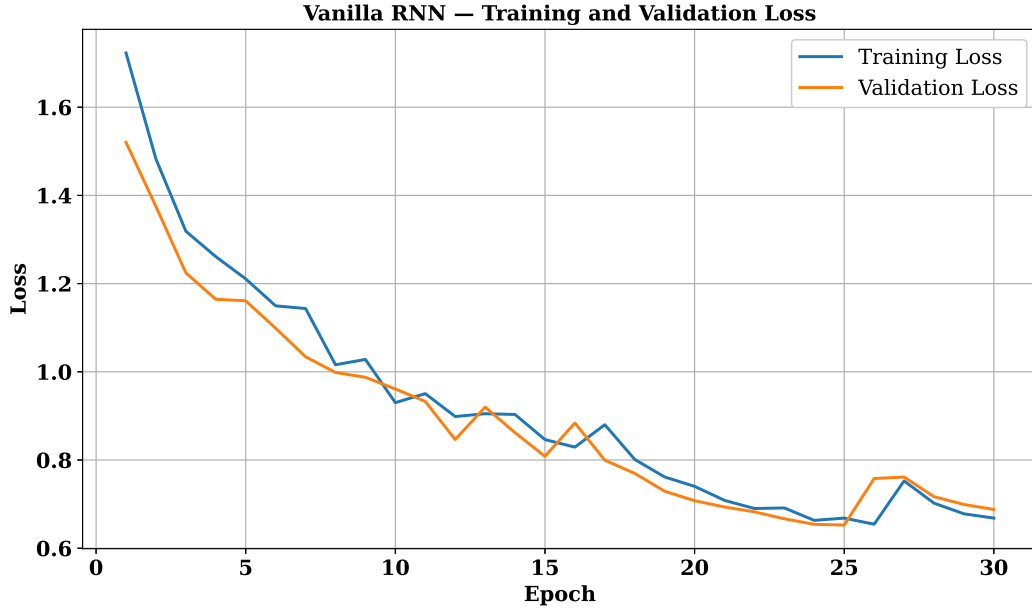


Figure 5: Training and validation loss of the Vanilla RNN.

The corresponding accuracy curves are shown in Figure 6.

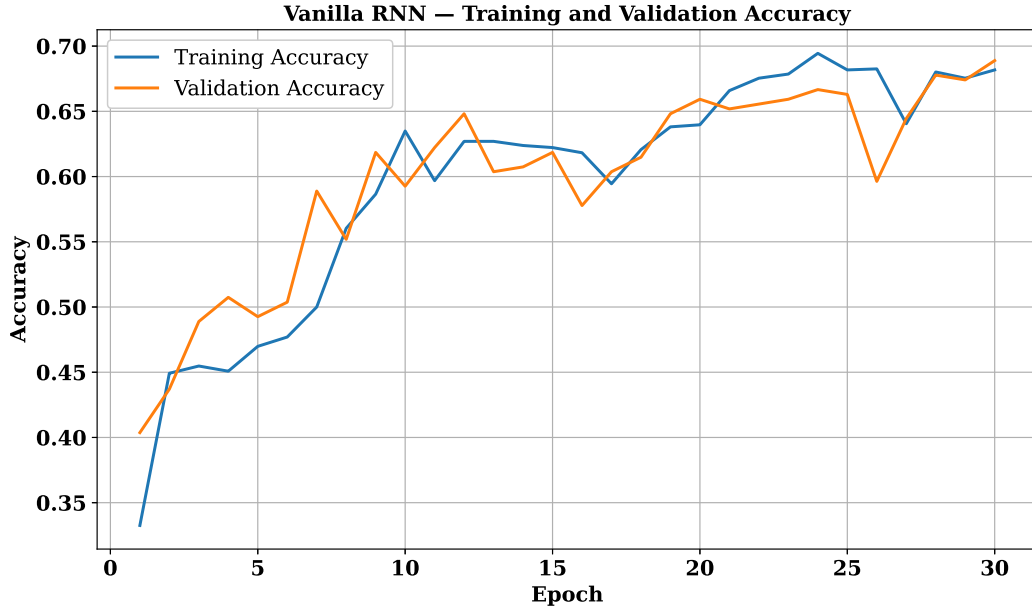


Figure 6: Training and validation accuracy of the Vanilla RNN.

The model required approximately 28.97 seconds for training in the latest recorded comparison run. The final evaluation was performed on the official UCI HAR test set containing 2947 samples.

Performance Evaluation

The Vanilla RNN was evaluated using accuracy, macro precision, macro recall and macro F1-score. Macro averaging was used so that the performance across all six activity classes was considered equally.

Metric	Vanilla RNN
Test Accuracy	65.22%
Macro Precision	68.03%
Macro Recall	63.86%
Macro F1-score	63.19%
Trainable Parameters	1,974
Training Time	28.97 s

The Vanilla RNN achieved a test accuracy of 65.22% and a macro F1-score of 63.19%. The difference between the two values shows that the model did not perform equally well for all activity classes.

This result is used as the baseline for comparing the LSTM and GRU models. Unlike those gated models, the basic RNN does not have a separate cell state or gates to control the information passed through the sequence.

Confusion Matrix Analysis

The confusion matrix was used to examine which activities were correctly classified and which activities were confused with one another.

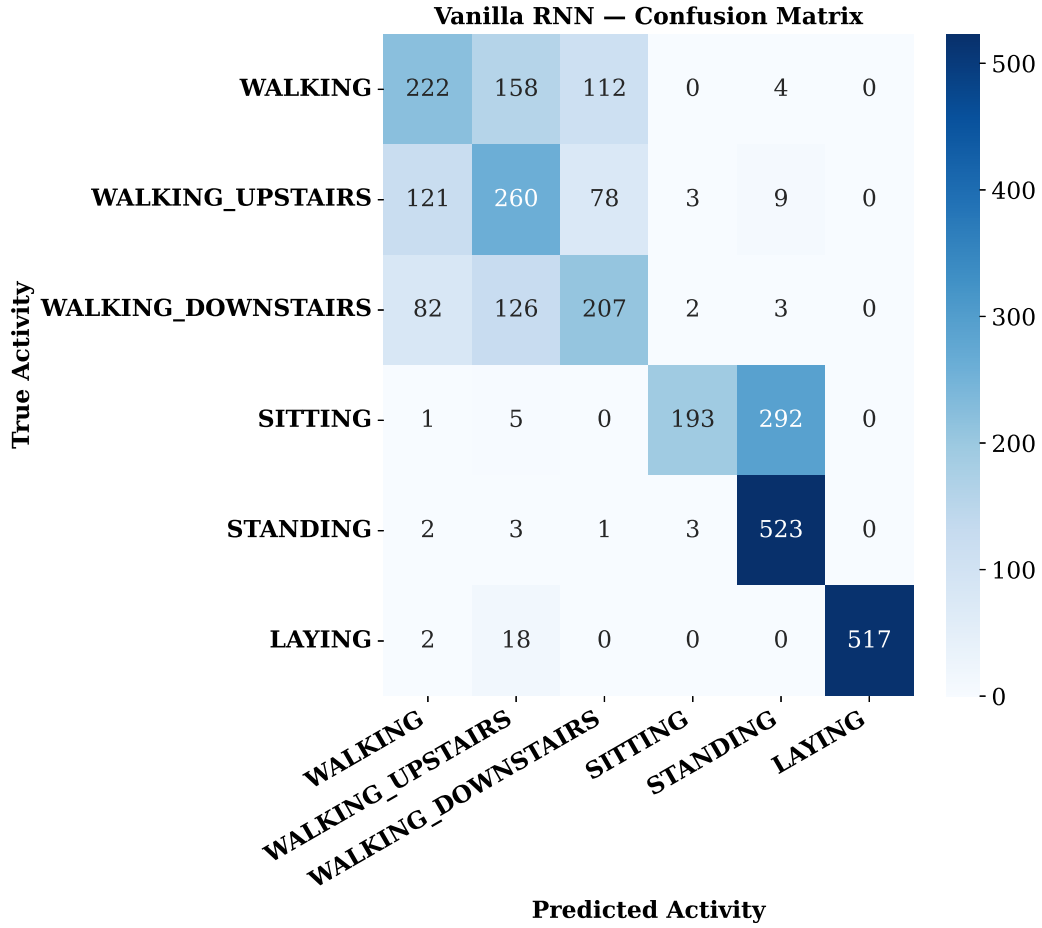


Figure 7: Confusion matrix of the Vanilla RNN on the official UCI HAR test set.

The confusion matrix gives more detail than the overall accuracy by showing how the model predicted each activity class. Some activities have similar sensor patterns, which can make them harder for the Vanilla RNN to distinguish. The errors are also not equally distributed across all six classes. Therefore, macro precision, recall and F1-score are useful along with accuracy for evaluating the model.

Overall, the Vanilla RNN was able to learn temporal patterns from the sensor data, but its performance was lower than the gated models used later. This leads to the next experiments with LSTM and GRU.

10. Long Short-Term Memory (LSTM) Architecture

Long Short-Term Memory (LSTM) is a gated recurrent architecture designed to retain useful information over a sequence while controlling which information should be added, removed or exposed.

Unlike the Vanilla RNN, the LSTM maintains two states at every time step:

$$h_t$$

which is the hidden state, and

$$c_t$$

which is the cell state.

The cell state provides a separate pathway through which information can be carried across time steps.

The LSTM used in this experiment contained 32 recurrent units. The classifier architecture was:

Input(128, 9) → LSTM(32) → Dropout(0.2) → Dense(16, ReLU) → Dense(6, Softmax)

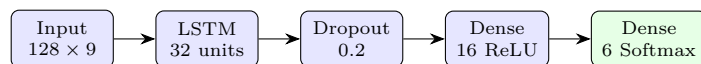


Figure 8: Architecture of the LSTM-based activity classifier.

LSTM Gates

At each time step, the LSTM uses three gates to regulate the information stored in the cell state. The forget gate determines which information from the previous cell state should be retained:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f).$$

The input gate determines which new information should be stored:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i).$$

A candidate cell-state value is calculated as:

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c).$$

The cell state is then updated using:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Finally, the output gate controls the information exposed as the new hidden state:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

and

$$h_t = o_t \odot \tanh(c_t).$$

Here, σ represents the sigmoid activation and $\odot$ represents element-wise multiplication.

The main difference between the LSTM and the Vanilla RNN is the use of a separate cell state along with gating mechanisms. The forget, input and output gates control how information is retained, added and passed through the sequence.

LSTM Implementation

The LSTM model was trained using the same preprocessed HAR dataset and classification setup used for the Vanilla RNN. Keeping the input data, optimizer settings and classifier structure consistent allowed the effect of replacing the recurrent layer to be examined.

The training configuration was:

Parameter	Setting
Recurrent layer	LSTM
Hidden units	32
Dropout	0.20
Dense layer	16 neurons, ReLU
Output layer	6 neurons, Softmax
Optimizer	Adam
Learning rate	0.001
Batch size	32
Maximum epochs	30

The model contained:

6,006 trainable parameters

The training and validation loss curves are shown below.

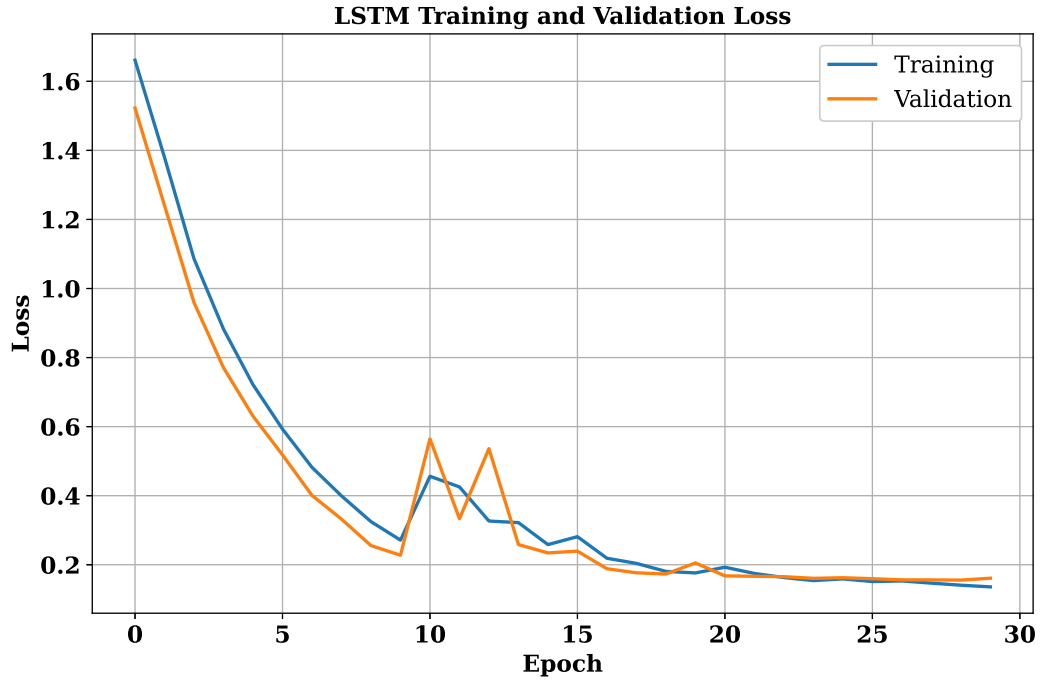


Figure 9: Training and validation loss of the LSTM model.

The corresponding accuracy curves are shown in Figure 10.

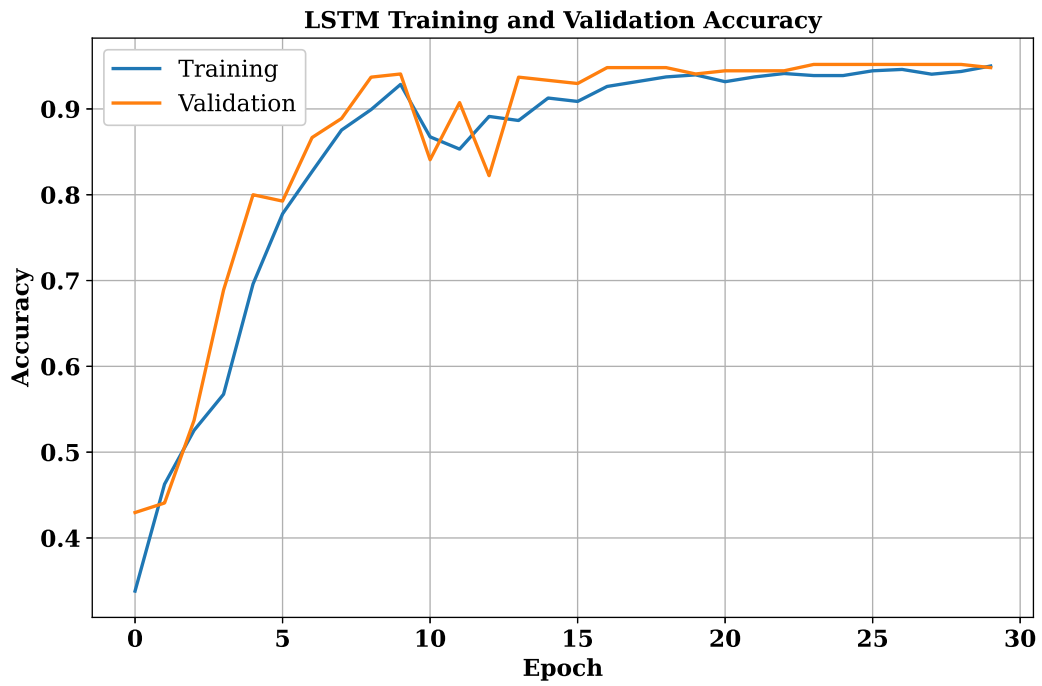


Figure 10: Training and validation accuracy of the LSTM model.

The model reached its best validation performance at the recorded best epoch, and training was stopped after the validation performance stopped improving. The training time for the latest run was approximately 20.70 seconds. For the final evaluation, the official UCI HAR test set containing 2947 samples was used.

LSTM Performance Evaluation

The LSTM model was evaluated on the official UCI HAR test set using accuracy, macro precision, macro recall and macro F1-score.

Metric	LSTM
Test Accuracy	87.00%
Macro Precision	87.11%
Macro Recall	86.68%
Macro F1-score	86.78%
Trainable Parameters	6,006
Training Time	20.70 s

The LSTM achieved a test accuracy of 87.00% and a macro F1-score of 86.78%. Compared with the Vanilla RNN, the LSTM showed a clear improvement in test performance in this experiment. This improvement is related to the LSTM gates, which control the flow of information through the sequence. The separate cell state also helps carry information across multiple time steps.

LSTM Confusion Matrix Analysis

The class-wise predictions of the LSTM were examined using the confusion matrix.

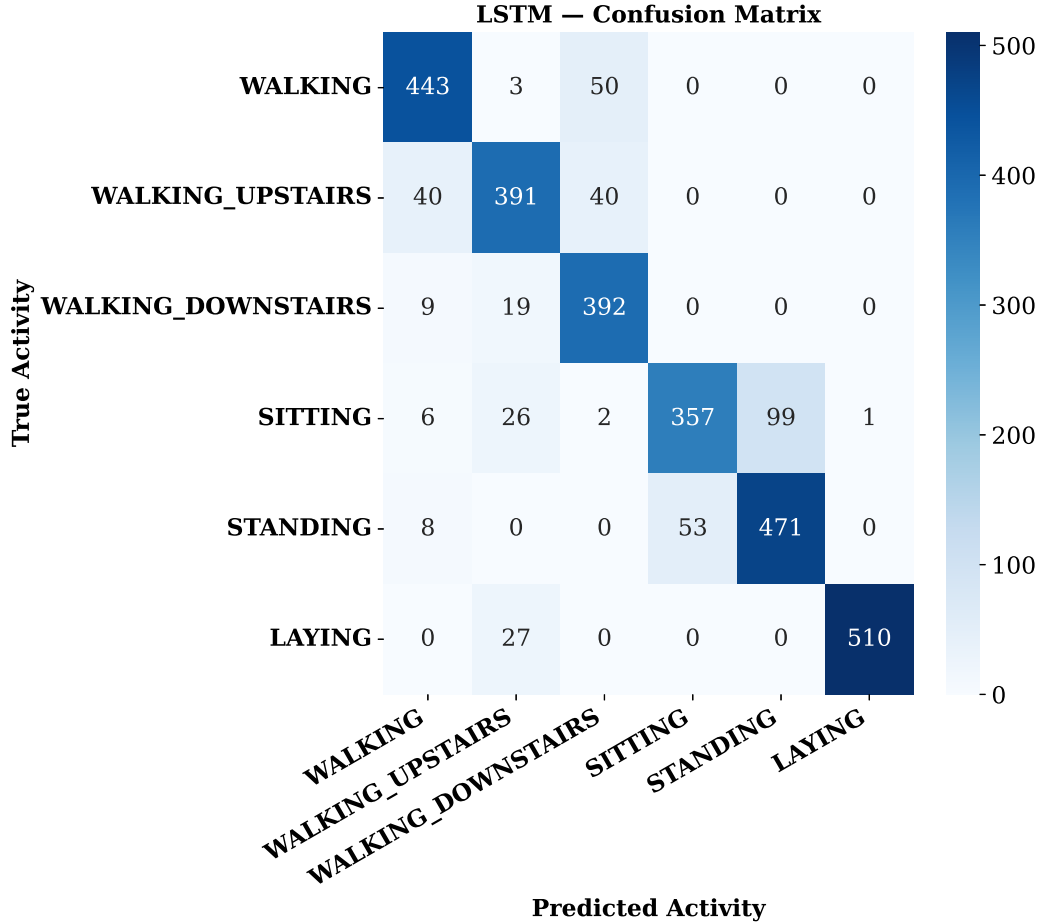


Figure 11: Confusion matrix of the LSTM model on the official UCI HAR test set.

The confusion matrix shows that the LSTM correctly classifies most samples across the activity classes. Some errors are still present between activities with similar sensor patterns.

The SITTING and STANDING classes show some confusion, which can be expected because both are relatively static activities with low body movement compared with activities such as WALKING.

The LAYING class is also classified correctly for most samples. Overall, the LSTM gives more balanced class-wise results than the Vanilla RNN, which is reflected in its higher macro F1-score.

Observation

Under the experimental configuration used in this study, the LSTM achieved higher test accuracy and macro F1-score than the Vanilla RNN.

The comparison was:

Metric	Vanilla RNN	LSTM
Accuracy	65.22%	87.00%
Macro Precision	68.03%	87.11%
Macro Recall	63.86%	86.68%
Macro F1	63.19%	86.78%
Parameters	1,974	6,006
Training Time	28.97 s	20.70 s

In this experiment, the LSTM showed a substantial improvement in test performance after introducing the gating mechanism. It also had more trainable parameters because of the additional gates and cell-state computations.

11. Gated Recurrent Unit (GRU) Architecture

The Gated Recurrent Unit (GRU) is a gated recurrent architecture that controls the flow of information using two gates: the reset gate and the update gate. Unlike the LSTM, the GRU does not maintain a separate cell state. Instead, the information is stored directly in the hidden state. The two gates allow the model to control how much previous information is retained and how much new information is incorporated.

The GRU architecture used in this experiment was:

Input(128, 9) $\rightarrow$ GRU(32) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(6, Softmax)

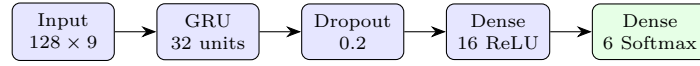


Figure 12: Architecture of the GRU-based activity classifier.

GRU Gates

The reset gate controls the contribution of the previous hidden state when calculating the candidate state:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r).$$

The update gate controls the balance between the previous hidden state and the candidate hidden state:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z).$$

The candidate hidden state is calculated as:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h).$$

The new hidden state is then obtained by combining the previous state and the candidate state:

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}.$$

Here, σ denotes the sigmoid activation and $\odot$ denotes element-wise multiplication.

The reset gate therefore controls how strongly the previous state contributes to the candidate state, while the update gate controls how much of the previous state is retained in the new hidden state.

Compared with the LSTM, the GRU uses fewer gates and does not maintain a separate cell state. This results in a smaller parameter count for the same number of recurrent units.

GRU Implementation

The GRU model was trained using the same preprocessed HAR dataset and the same classification pipeline used for the Vanilla RNN and LSTM models.

The configuration was:

Parameter	Setting
Recurrent layer	GRU
Hidden units	32
Dropout	0.20
Dense layer	16 neurons, ReLU
Output layer	6 neurons, Softmax
Optimizer	Adam
Learning rate	0.001
Batch size	32
Maximum epochs	30

The model contained:

4,758 trainable parameters

The training and validation loss are shown in Figure 13.

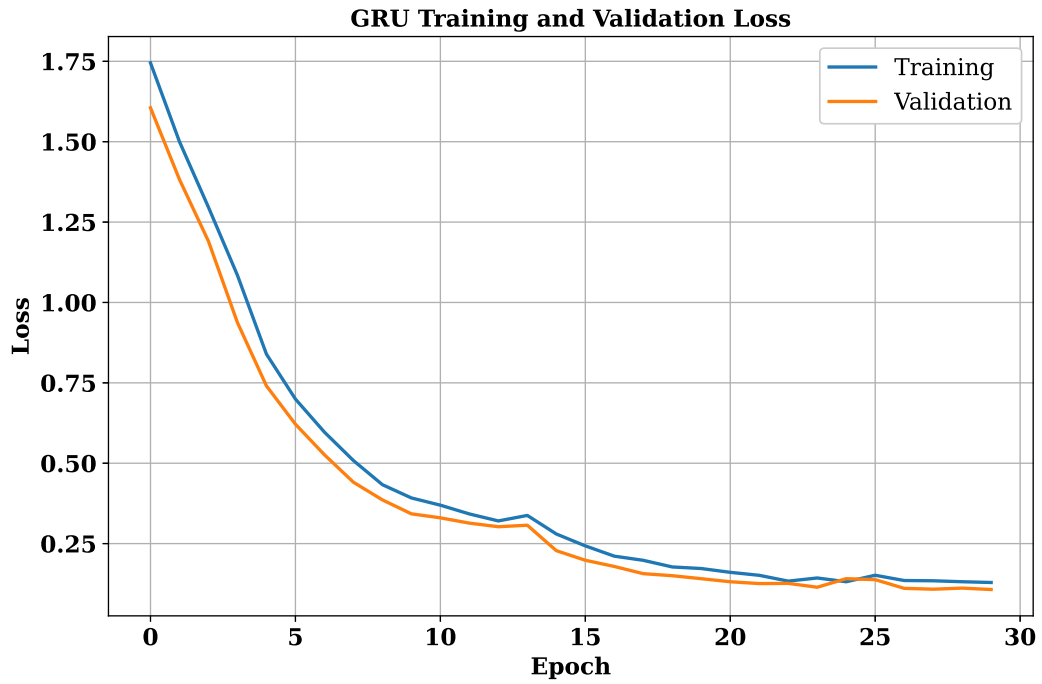


Figure 13: Training and validation loss of the GRU model.

The corresponding training and validation accuracy curves are shown below.

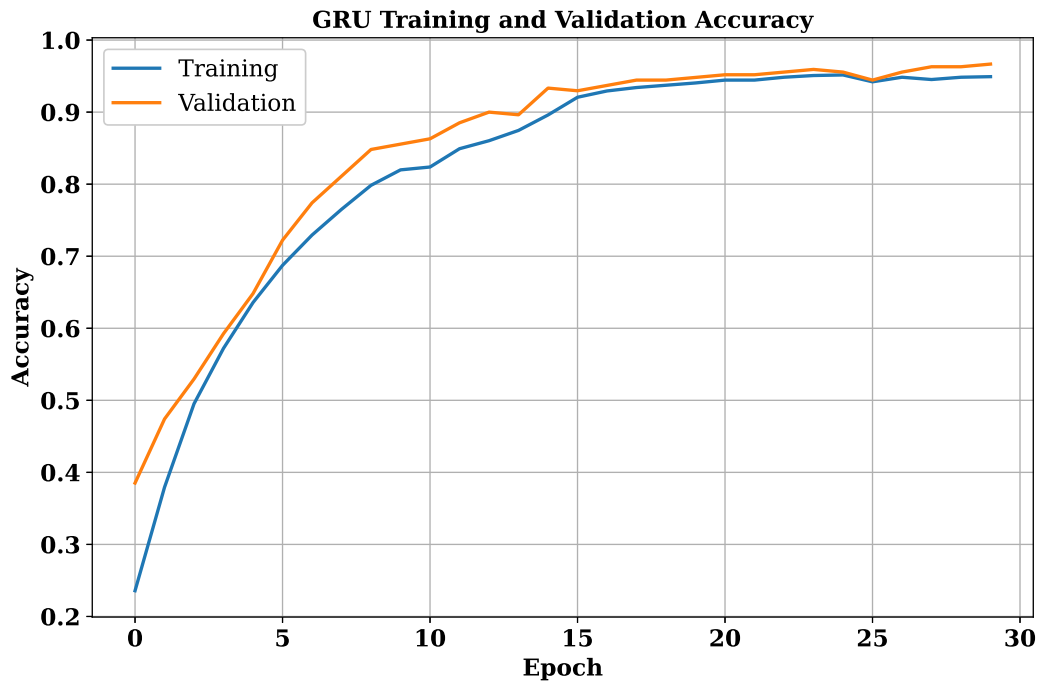


Figure 14: Training and validation accuracy of the GRU model.

The latest recorded training run required approximately 19.89 seconds.

The trained model was finally evaluated on the official UCI HAR test set containing 2947 samples.

GRU Performance Evaluation

The GRU model was evaluated on the official UCI HAR test set using accuracy, macro precision, macro recall and macro F1-score.

Metric	GRU
Test Accuracy	89.01%
Macro Precision	89.38%
Macro Recall	88.97%
Macro F1-score	88.96%
Trainable Parameters	4,758
Training Time	19.89 s

The GRU achieved a test accuracy of 89.01% and a macro F1-score of 88.96%. In this experiment, these values were higher than those obtained by both the Vanilla RNN and LSTM.

The GRU also used fewer trainable parameters than the LSTM while still using a gated recurrent structure. These results allow the three models to be compared in terms of both classification performance and number of trainable parameters.

GRU Confusion Matrix Analysis

The class-wise predictions of the GRU were examined using the confusion matrix.

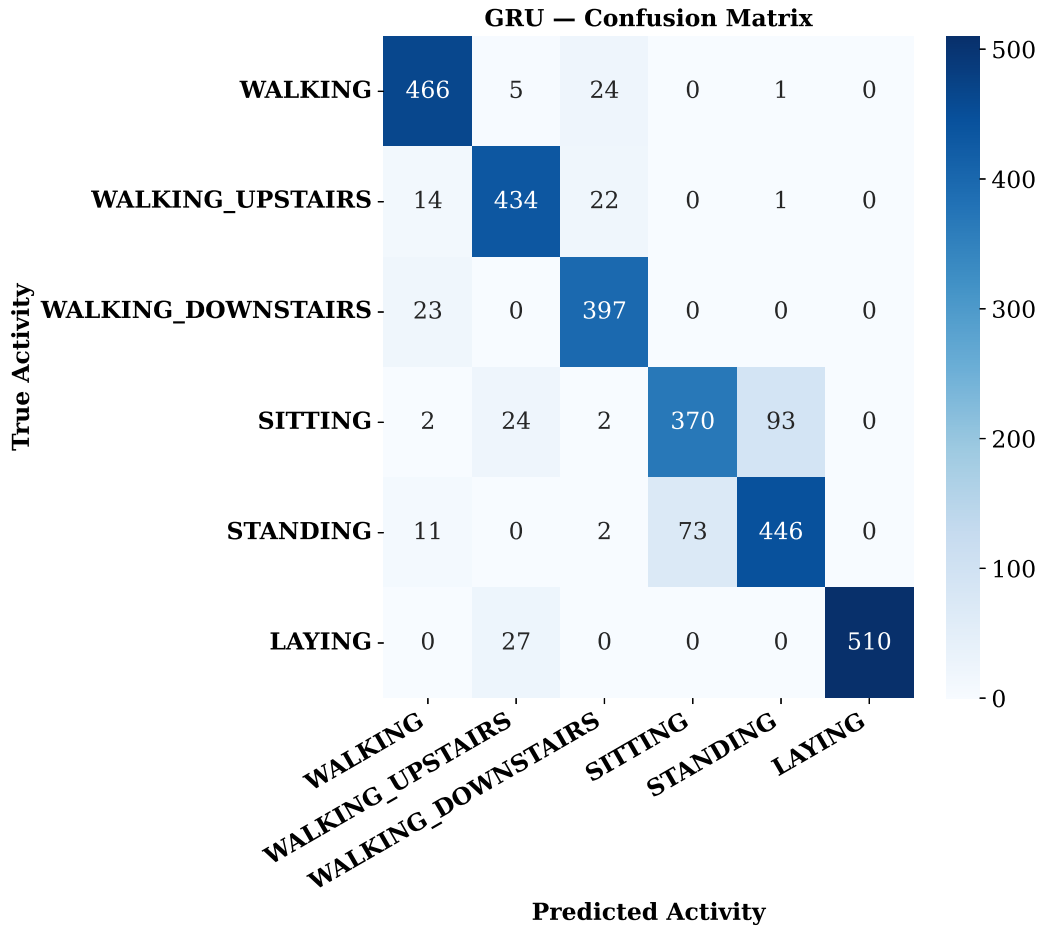


Figure 15: Confusion matrix of the GRU model on the official UCI HAR test set.

The confusion matrix shows that the GRU correctly classifies most samples across the six activity classes. The remaining errors are mainly between activities with similar sensor patterns.

Some confusion is seen between SITTING and STANDING, similar to the other recurrent models. Errors are also present among WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS because these activities have related motion patterns.

The confusion matrix therefore gives a clearer view of where the GRU makes classification errors in addition to the overall accuracy.

12. RNN vs LSTM vs GRU Comparison

The three recurrent architectures were evaluated using the same HAR classification problem. The main quantitative results are summarized below.

Metric	RNN	LSTM	GRU
Accuracy	65.22%	87.00%	89.01%
Macro Precision	68.03%	87.11%	89.38%
Macro Recall	63.86%	86.68%	88.97%
Macro F1	63.19%	86.78%	88.96%
Parameters	1,974	6,006	4,758
Training Time	28.97 s	20.70 s	19.89 s

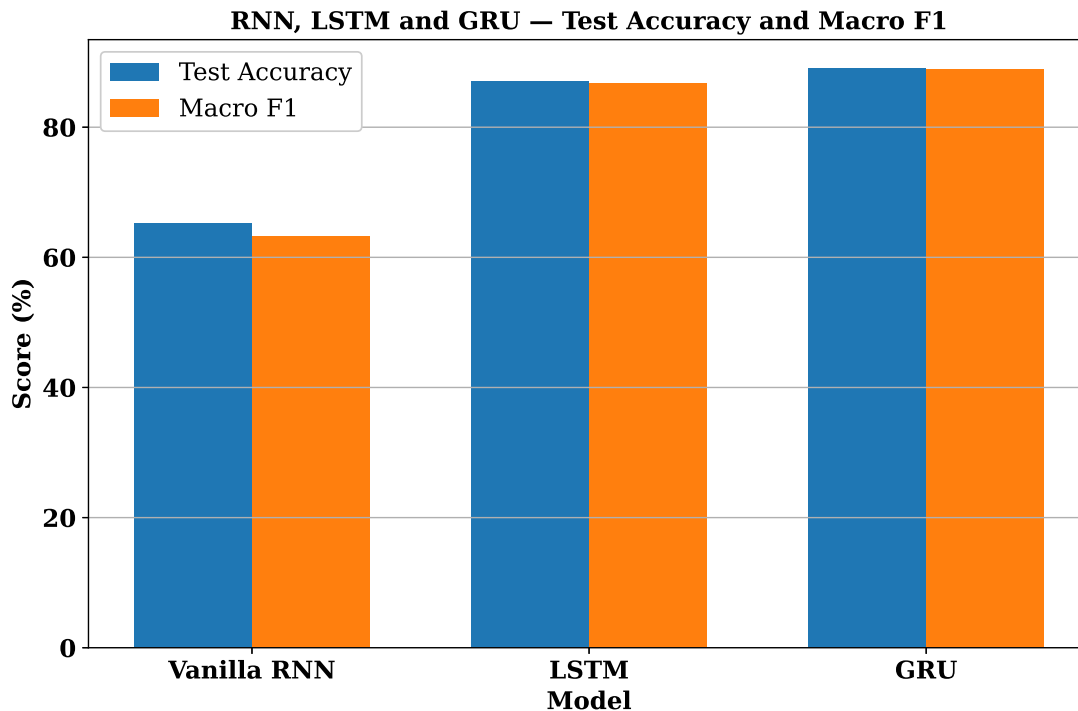


Figure 16: Comparison of test accuracy and macro F1-score for RNN, LSTM and GRU.

The Vanilla RNN obtained the lowest test accuracy and macro F1-score among the three models. Both LSTM and GRU showed higher test performance under the same experimental setup.

The GRU achieved 89.01% test accuracy and an 88.96% macro F1-score, while the LSTM achieved 87.00% accuracy and an 86.78% macro F1-score.

The models also differed in their number of trainable parameters. The LSTM had 6,006 parameters, the GRU had 4,758 parameters, and the Vanilla RNN had 1,974 parameters.

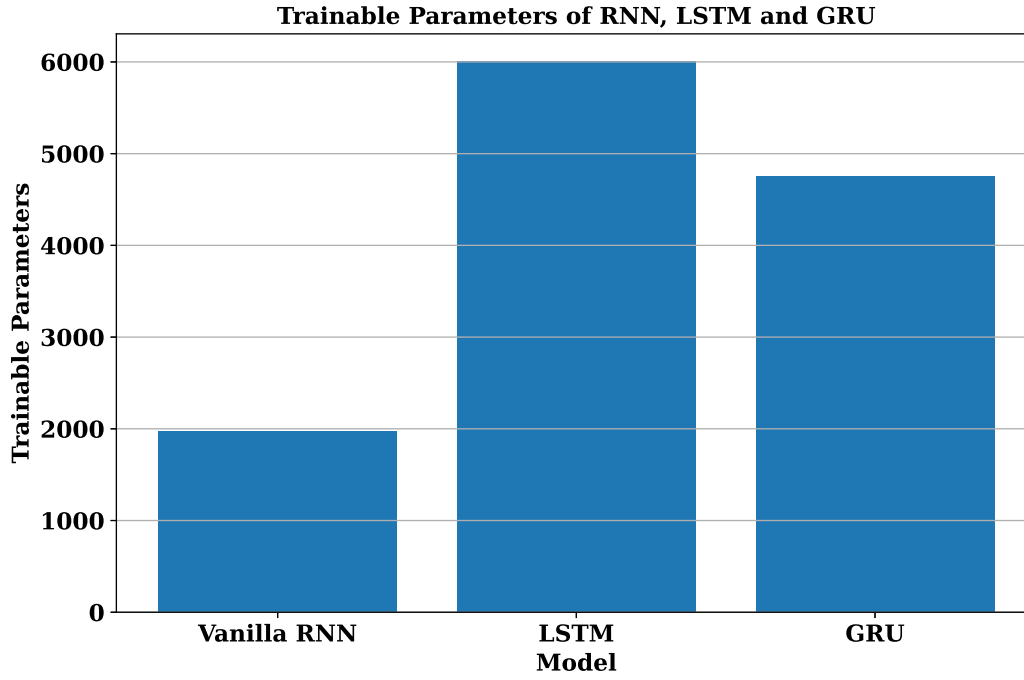


Figure 17: Trainable parameter count of the three recurrent models.

The LSTM therefore had the largest number of trainable parameters because of its multiple gates and separate cell-state mechanism. The GRU used fewer parameters than the LSTM while retaining a gated recurrent structure. The measured training times were:

Model	Training Time
RNN	28.97 s
LSTM	20.70 s
GRU	19.89 s

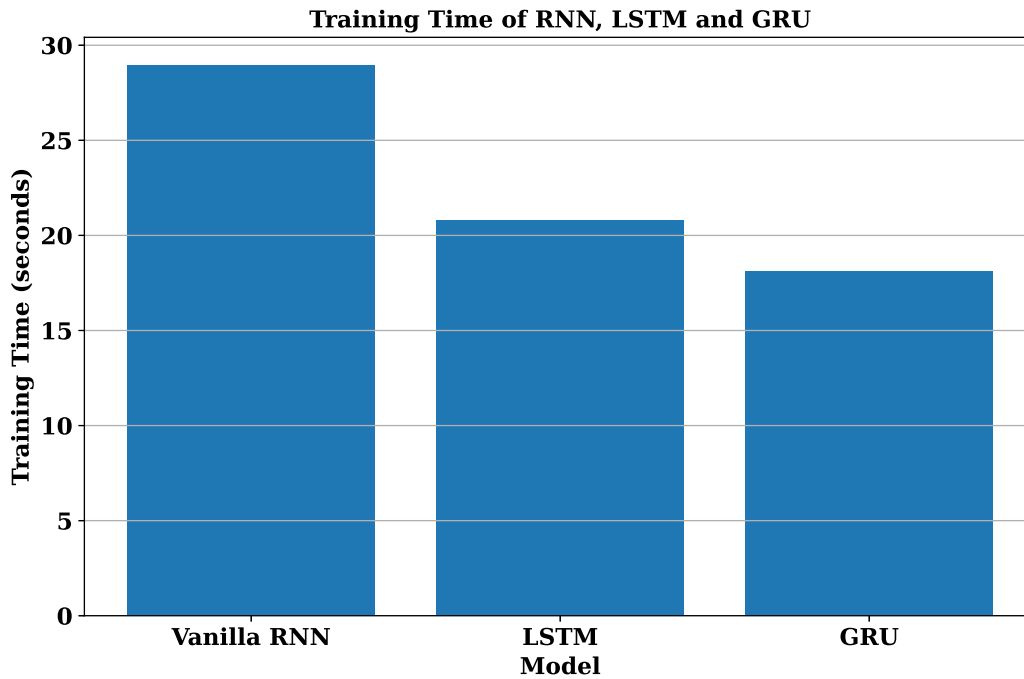


Figure 18: Training time comparison for RNN, LSTM and GRU.

In this run, the GRU had the lowest recorded training time, while the LSTM took slightly longer. The Vanilla RNN took the longest time even though it had the fewest parameters. This shows that training time depends on the actual execution environment and cannot be determined from the number of parameters alone.

Architectural Comparison

The main structural differences between the three recurrent architectures are summarized below.

Feature	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Separate cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Reset gate	No	No	Yes
Update gate	No	No	Yes
Number of recurrent units	32	32	32
Trainable parameters	1,974	6,006	4,758

The Vanilla RNN has the simplest recurrent structure. The LSTM adds a separate cell state and three gates, while the GRU uses two gates and stores the information directly in the hidden state.

These architectural differences were reflected in the performance and computational results obtained on the HAR dataset.

13. Effect of Sequence Length

The effect of sequence length was studied by evaluating the three recurrent models using sequence lengths of 32, 64 and 128 time steps. The purpose was to observe whether providing a shorter or longer temporal context affected the classification performance.

The macro F1-score obtained at each sequence length is shown below.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	68.22%	84.83%	84.69%
64	68.78%	85.46%	89.08%
128	63.19%	86.78%	88.96%

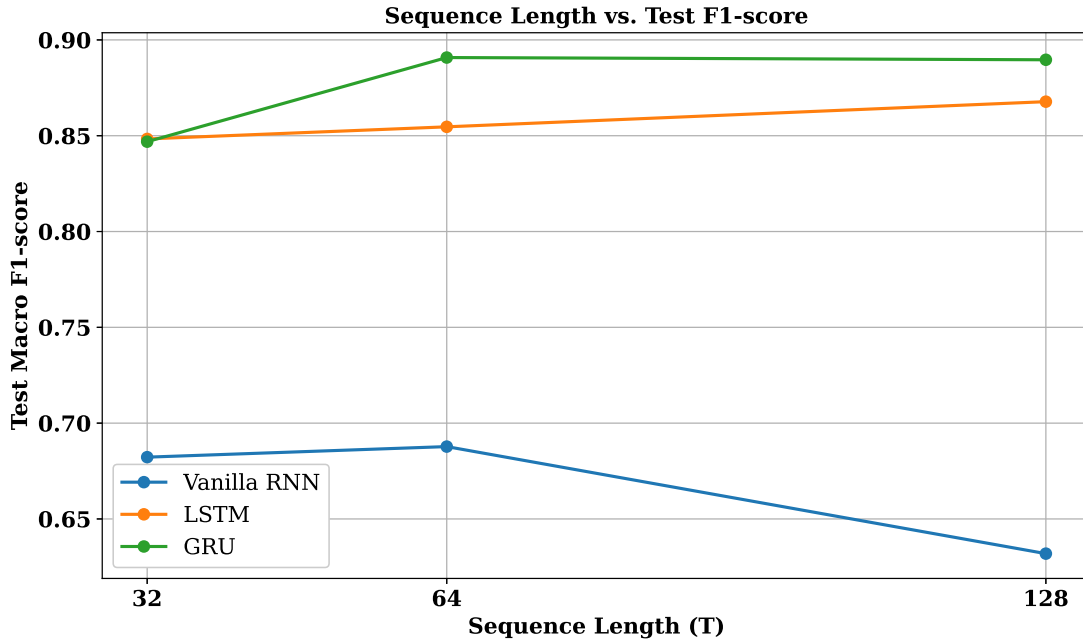


Figure 19: Effect of sequence length on test macro F1-score for RNN, LSTM and GRU.

The Vanilla RNN obtained F1-scores of 68.22%, 68.78% and 63.19% for sequence lengths of 32, 64 and 128, respectively. Its highest F1-score was at 64 time steps, while the performance decreased at 128 time steps.

The LSTM showed a more consistent increase, with F1-scores of 84.83%, 85.46% and 86.78% for 32, 64 and 128 time steps, respectively. In this experiment, the highest LSTM F1-score was obtained at 128 time steps.

The GRU achieved 84.69% at 32 time steps, increased to 89.08% at 64 time steps and slightly decreased to 88.96% at 128 time steps. Its highest measured F1-score was therefore at 64 time steps.

These results show that changing the sequence length did not affect all three models in the same way. The RNN performance decreased at 128 time steps, while the LSTM continued to improve. The GRU reached its highest value at 64 time steps and changed only slightly at 128.

Overall, the effect of sequence length depended on the recurrent architecture and the amount of temporal information provided to the model.

Sequence Length and Computational Cost

The training time was also recorded for each sequence length. The measured values are shown below.

Sequence Length	RNN	LSTM	GRU
32	23.46 s	11.33 s	15.35 s
64	17.58 s	16.39 s	16.62 s
128	23.74 s	19.46 s	18.06 s

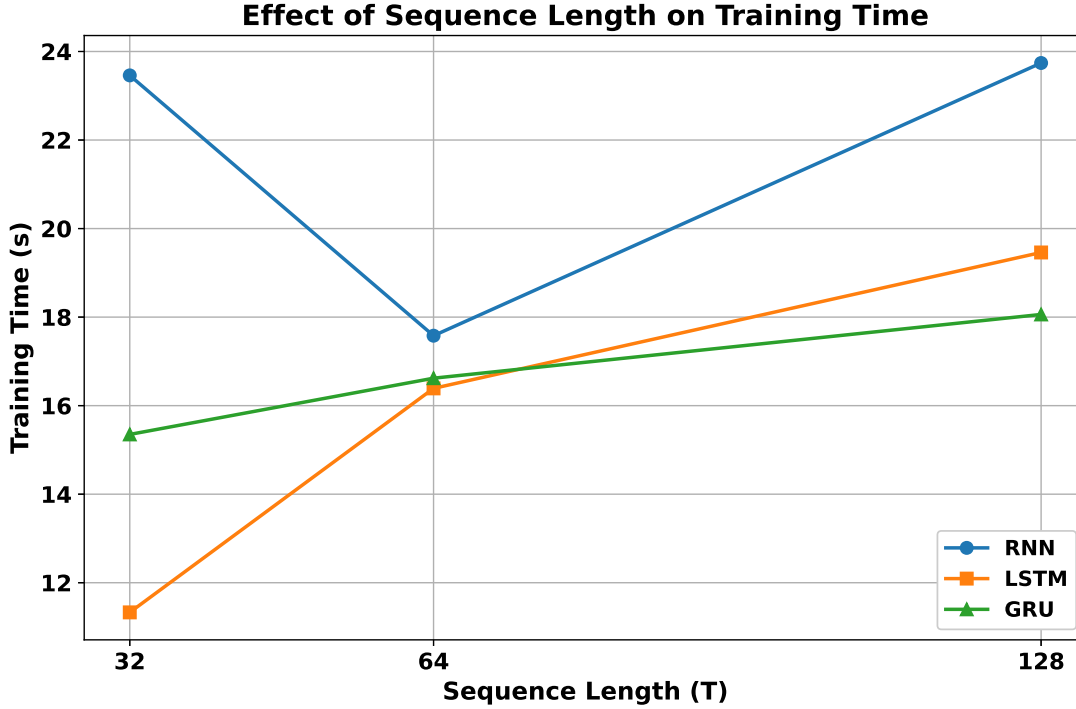


Figure 20: Measured training time for different sequence lengths.

From sequence length 32 to 128, the training time increased by 8.13 seconds for the LSTM and by 2.71 seconds for the GRU. For the RNN, the change was small, from 23.46 seconds to 23.74 seconds.

The training time did not increase consistently with sequence length. For example, the RNN took 23.46 seconds at 32 time steps but only 17.58 seconds at 64 time steps. This shows that the measured training time is also affected by factors such as batching, framework execution and the computing environment.

A longer sequence still requires the recurrent layer to process more time steps, increasing the amount of temporal computation. This was more clearly reflected in the LSTM and GRU timings in the recorded results.

Therefore, sequence length should be evaluated using both model performance and measured training time rather than assuming that a longer sequence will always give better performance or proportionally higher training time.

Inference

The sequence-length experiment showed that no single sequence length gave the highest F1-score for all three architectures.

For the RNN, the highest measured F1-score was at 64 time steps. The LSTM F1-score increased as the sequence length increased, reaching its highest value at 128 time steps. For the GRU, the highest measured F1-score was also at 64 time steps.

This shows that the useful temporal context depends on the recurrent architecture. In this experiment, the LSTM benefited from the longest tested sequence, while the RNN and GRU achieved their highest measured F1-score at 64 time steps.

14. Video Understanding using CNN and RNN

The second part of the experiment extended sequence learning from sensor signals to video data. A video contains both spatial information within individual frames and temporal information across successive frames. Therefore, a

CNN was used to extract spatial features from individual frames, while a recurrent network was used to learn the temporal relationship between those features.

A three-class subset of the UCF101-derived video dataset was used for this experiment:

- Archery
- Basketball
- BaseballPitch

The dataset was divided into training, validation and test videos. The training and validation sets contained 30 and 3 videos per class respectively. The test set contained 28 videos in total:

Class	Training	Validation	Test
Archery	30	3	7
Basketball	30	3	11
BaseballPitch	30	3	10
Total	90	9	28

The video processing pipeline was:

Video $\rightarrow$ Frame Sampling $\rightarrow$ CNN Feature Extraction $\rightarrow$ Feature Sequence $\rightarrow$ LSTM/GRU $\rightarrow$ Class Prediction

Ten frames were uniformly sampled from each video. Each frame was resized to 224×224 pixels and represented as an RGB image.

The extracted CNN features were then arranged as a temporal sequence:

$$10 \times 1280.$$

Thus, each video was represented as ten consecutive 1280-dimensional feature vectors.

Expected Video Input and Output

The input to the video pipeline was a video containing one of the three selected human activities.

For each video:

$$\text{Number of sampled frames} = 10$$

$$\text{Frame size} = 224 \times 224 \times 3$$

The CNN converted each frame into a feature vector of dimension 1280. Therefore, after feature extraction, the recurrent network received:

$$X_{\text{video}} \in \mathbb{R}^{10 \times 1280}.$$

The output was a three-class probability vector:

$$\hat{y} \in \mathbb{R}^3.$$

The predicted class was selected using the maximum output probability.

Video Frame Sampling

Ten frames were sampled sequentially from each video to represent its temporal content. A sequential decoding procedure was used because the metadata-based random seeking of some AVI files was unreliable. The videos were successfully decoded and the sampled frames were resized to 224×224 before CNN feature extraction.

A representative set of sampled frames is shown below.

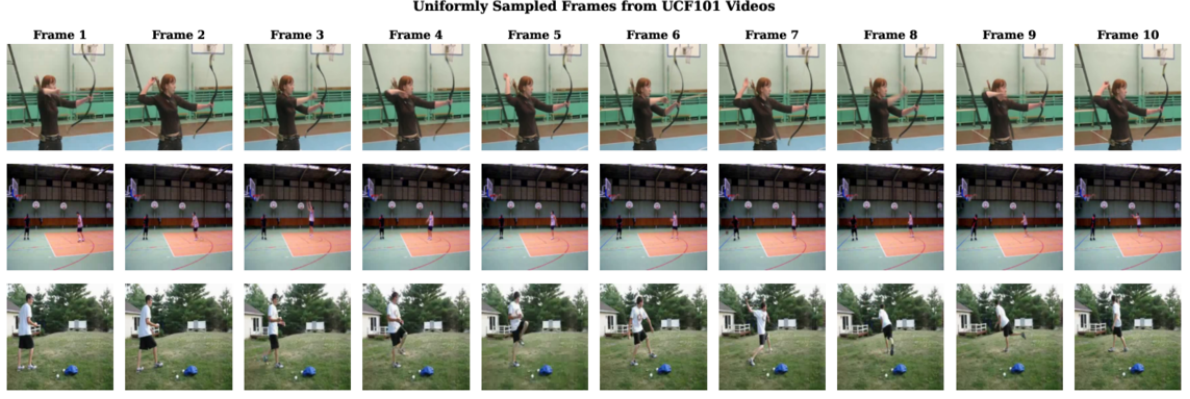


Figure 21: Representative frames sampled from the selected video classes.

The figure shows that a video contains multiple frames representing the same action. Using only one frame would miss the temporal information, while the sequence of frame-level features allows the recurrent model to learn how the visual features change over time.

15. CNN Feature Extraction

A pretrained MobileNetV2 model with ImageNet weights was used as the spatial feature extractor. The classification head was removed and global average pooling was used to obtain a compact feature vector for each frame. The CNN processing was:

$$224 \times 224 \times 3 \rightarrow \text{MobileNetV2} \rightarrow \text{Global Average Pooling} \rightarrow 1280\text{-D feature vector.}$$

The CNN parameters were kept frozen during the recurrent-model training. Therefore, the CNN was responsible for extracting spatial information from individual frames, while the recurrent layer learned the temporal relationship between the extracted frame features.

For ten sampled frames, the resulting feature sequence was:

$$10 \times 1280.$$

This sequence was then supplied to either an LSTM or GRU layer.

The overall feature-extraction process can therefore be represented as:

$$I_1, I_2, \dots, I_{10} \rightarrow f_1, f_2, \dots, f_{10} \rightarrow [f_1, f_2, \dots, f_{10}]$$

where each I_t is an input video frame and each $f_t \in \mathbb{R}^{1280}$ is its CNN feature vector.

16. CNN–LSTM / CNN–GRU Model

CNN–LSTM Architecture

The first video model combined the frozen MobileNetV2 feature extractor with an LSTM network.

The processing pipeline was:

$$\text{Video} \rightarrow 10 \text{ Frames} \rightarrow \text{MobileNetV2} \rightarrow 10 \times 1280 \rightarrow \text{LSTM} \rightarrow \text{Dense} \rightarrow 3\text{-class Softmax}$$

The recurrent network therefore received the CNN features in temporal order. The LSTM processed the ten feature vectors and generated a representation of the temporal information contained in the video.

The CNN–LSTM model contained:

$$168,643 \text{ trainable parameters}$$

The training and validation loss are shown below.

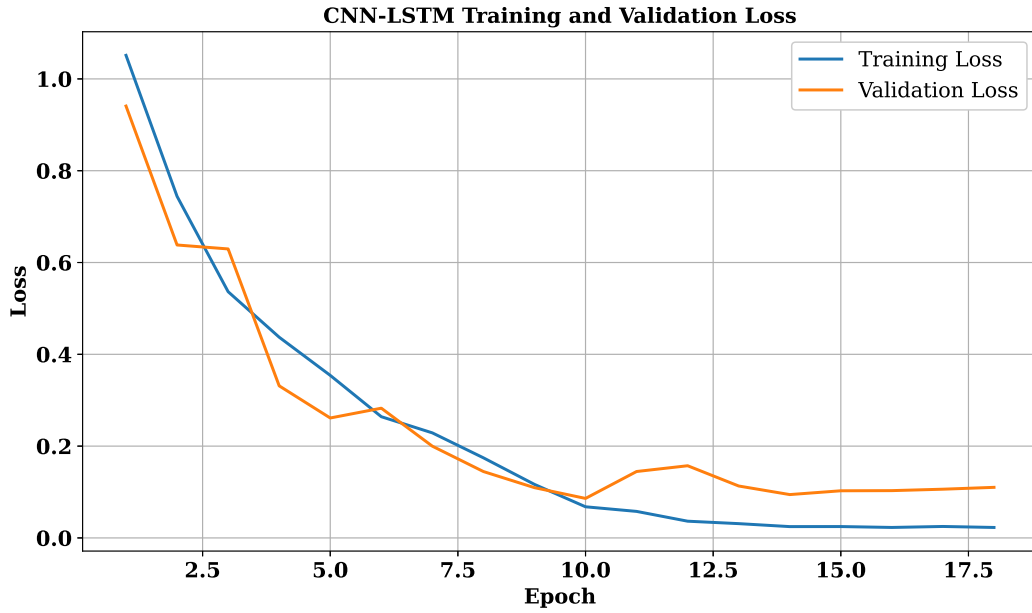


Figure 22: Training and validation loss of the CNN-LSTM video classifier.

The corresponding accuracy curves are shown below.

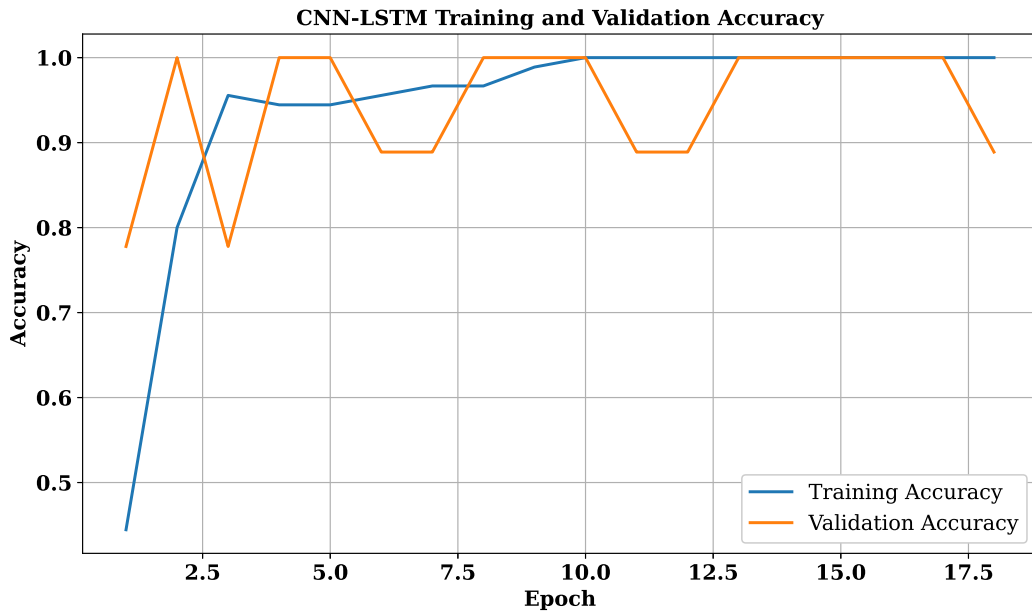


Figure 23: Training and validation accuracy of the CNN-LSTM video classifier.

The CNN-LSTM model achieved a test accuracy of:

96.43%

with a macro precision of 96.67%, macro recall of 96.97% and macro F1-score of 96.19%.

CNN-LSTM Confusion Matrix

The test predictions of the CNN-LSTM model were examined using a confusion matrix.

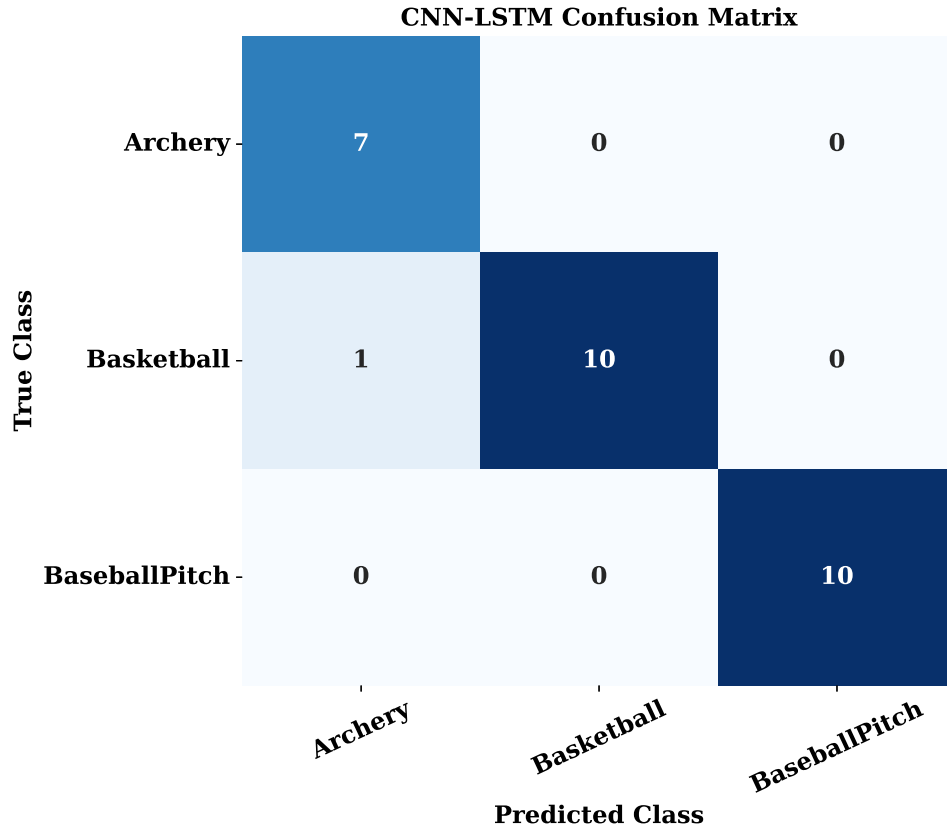


Figure 24: Confusion matrix of the CNN-LSTM video classifier.

The CNN-LSTM correctly classified most of the test videos. The confusion matrix shows that most predictions are on the diagonal, indicating correct classification for the three selected action classes.

The model achieved a macro F1-score of 96.19%, which is close to the test accuracy of 96.43%. This shows that the classification performance was relatively balanced across the three classes in this test set.

However, the test set contained only 28 videos. Therefore, the 96.43% accuracy represents the result for this particular test set and should not be generalized to the full UCF101 dataset.

CNN-GRU Architecture

The second video model replaced the LSTM with a GRU while keeping the CNN feature extraction pipeline unchanged.

The architecture was:

Video $\rightarrow$ 10 Frames $\rightarrow$ MobileNetV2 $\rightarrow$ 10×1280 $\rightarrow$ GRU $\rightarrow$ Dense $\rightarrow$ 3-class Softmax

The purpose of keeping the CNN feature extractor unchanged was to compare the temporal models under the same frame-level representation.

The CNN-GRU model contained:

126,723 trainable parameters

The training and validation loss are shown below.

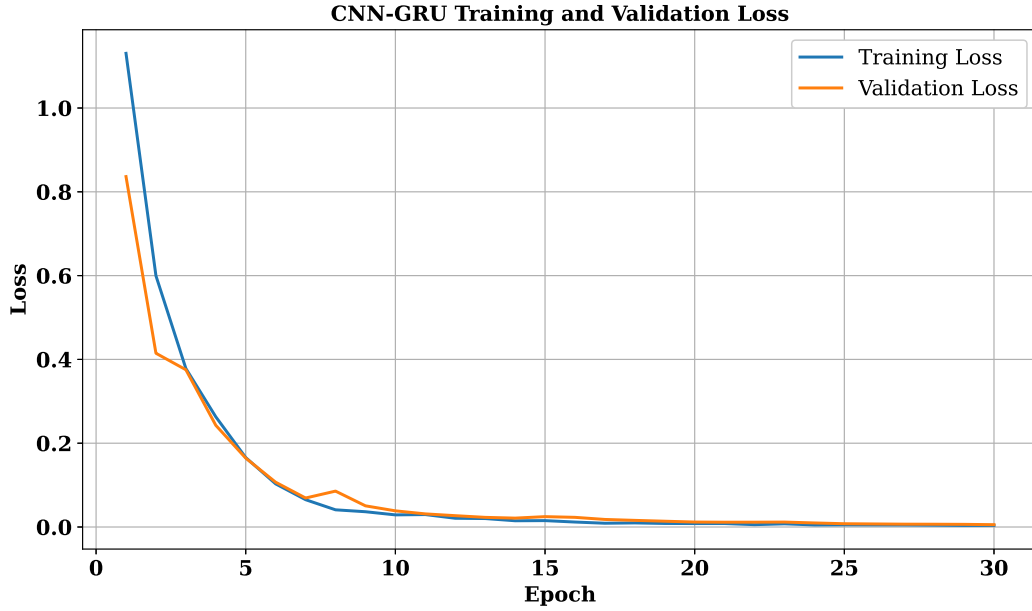


Figure 25: Training and validation loss of the CNN-GRU video classifier.

The corresponding accuracy curves are shown below.

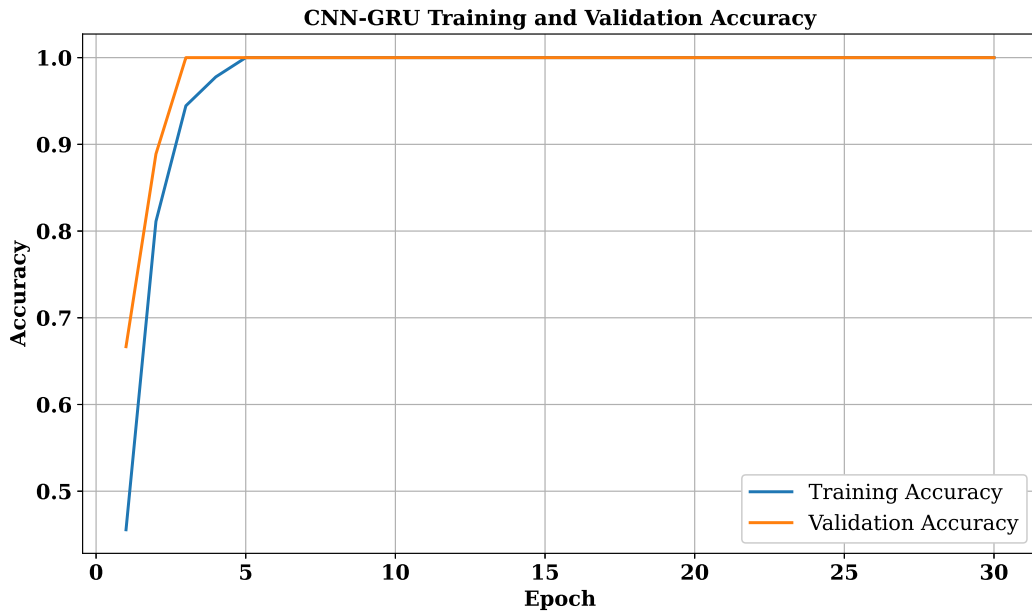


Figure 26: Training and validation accuracy of the CNN-GRU video classifier.

The CNN-GRU achieved a test accuracy of:

89.29%

with a macro precision of 89.56%, macro recall of 90.91% and macro F1-score of 88.85%.

CNN-GRU Confusion Matrix

The CNN-GRU predictions were analysed using the test confusion matrix.

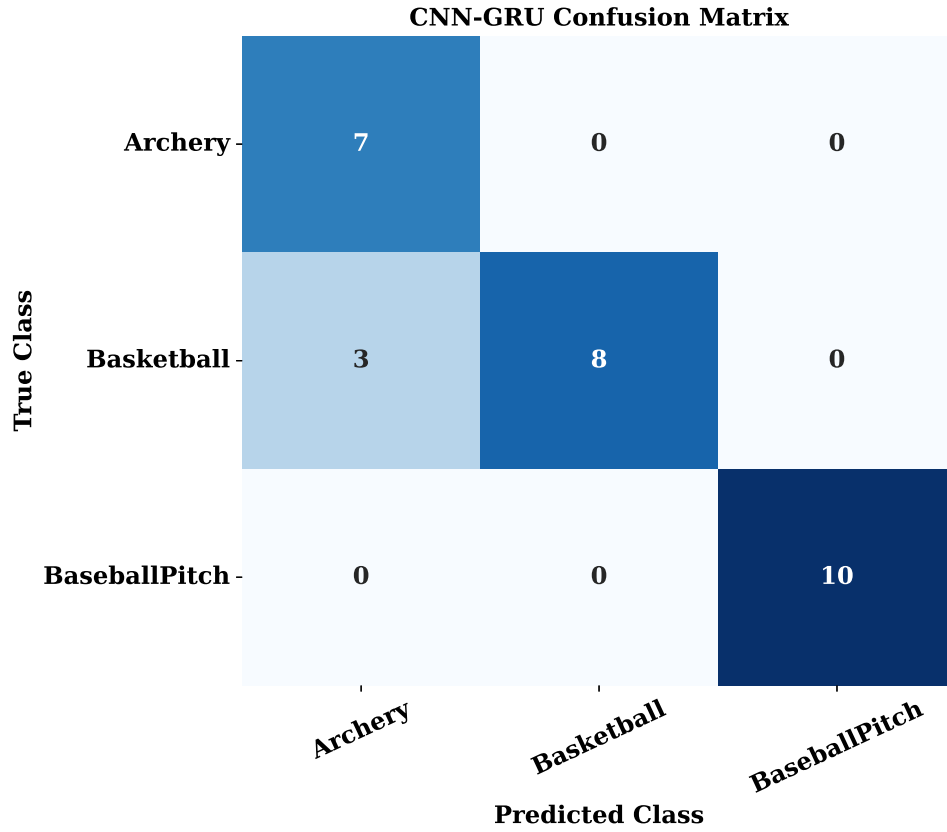


Figure 27: Confusion matrix of the CNN-GRU video classifier.

The confusion matrix shows that the CNN-GRU correctly classified all of the BaseballPitch test samples. Some errors occurred between Archery and Basketball, and between Basketball and BaseballPitch.

The class-wise results were:

Class	Precision	Recall	F1-score
Archery	77.78%	100.00%	87.50%
Basketball	100.00%	72.73%	84.21%
BaseballPitch	90.91%	100.00%	95.24%

The Basketball class had the lowest recall among the three classes. This indicates that some Basketball videos were assigned to other classes, which is also visible in the confusion matrix.

CNN-LSTM vs CNN-GRU Comparison

The two video models were compared using the same CNN feature extractor and the same three-class video dataset.

Metric	CNN-LSTM	CNN-GRU
Test Accuracy	96.43%	89.29%
Macro Precision	96.67%	89.56%
Macro Recall	96.97%	90.91%
Macro F1-score	96.19%	88.85%
Trainable Parameters	168,643	126,723
Training Time	10.30 s	10.30 s

The CNN-LSTM achieved higher test accuracy and macro F1-score than the CNN-GRU in this experiment. However, the CNN-GRU used fewer trainable parameters.

This difference in parameter count comes from the recurrent structures used after the CNN feature extractor. The LSTM has more gating parameters, while the GRU uses a simpler gated structure.

Both models had a recorded training time of approximately 10.30 seconds in this run. Therefore, there was no measurable difference in training time despite the difference in parameter count.

Inference

The video experiment demonstrated the separation of spatial and temporal processing.

The pretrained MobileNetV2 extracted a 1280-dimensional spatial feature vector from each sampled frame. These frame-level features were then arranged in temporal order and given to a recurrent network for temporal processing. The CNN-LSTM achieved 96.43% test accuracy and 96.19% macro F1-score, while the CNN-GRU achieved 89.29% test accuracy and 88.85% macro F1-score.

In this experiment, the CNN-LSTM showed higher measured classification performance, while the CNN-GRU used fewer trainable parameters. Since the test set contained only 28 videos, these results are limited to the implemented three-class experiment and should not be generalized to the full UCF101 dataset.

Overall, the experiment showed how a CNN can extract spatial features from individual video frames and a recurrent model can then learn the temporal relationships between those features.

17. Sequence-to-Sequence Learning

A sequence-to-sequence (Seq2Seq) model was implemented to study encoder-decoder based sequence learning. Unlike the previous classification experiments, the objective here was to generate an output sequence from an input sequence. A synthetic sequence reversal task was used. Each input consisted of four integer tokens, and the target output was the same sequence in reverse order.

For example:

$$[4, 2, 3, 7] \rightarrow [7, 3, 2, 4].$$

The dataset contained 5000 sequences. The data was divided into:

Partition	Samples
Training	3500
Validation	750
Test	750
Total	5000

The input sequence length was four. The token vocabulary contained the values 1–9 together with special padding, start and end tokens.

The special tokens were represented as:

$$\text{PAD} = 0, \quad \text{START} = 10, \quad \text{END} = 11.$$

Therefore, the total vocabulary size was:

$$\boxed{12}.$$

The model used an embedding dimension of 32 and an LSTM latent dimension of 64.

Seq2Seq Architecture

The Seq2Seq model consisted of an LSTM encoder followed by an LSTM decoder and a final dense layer with softmax activation.

The architecture can be represented as:

$$\boxed{\text{Input Sequence} \rightarrow \text{Embedding} \rightarrow \text{Encoder LSTM} \rightarrow \text{Context} \rightarrow \text{Decoder LSTM} \rightarrow \text{Dense Softmax} \rightarrow \text{Output Sequence}}$$

The encoder processes the complete input sequence and produces the recurrent state used to initialize the decoder. The decoder then generates the output sequence one token at a time.

The model contained:

$$\boxed{51,212 \text{ trainable parameters}}.$$

The training objective was to minimize the categorical sequence prediction loss over the output tokens.

Seq2Seq Training

The Seq2Seq model was trained on 3500 training sequences, while 750 sequences were used for validation and another 750 sequences were reserved for final testing.

The training configuration used an embedding dimension of 32 and an LSTM latent dimension of 64. The model was trained for a maximum of 30 epochs.

The best recorded validation loss was:

$$\boxed{0.001453}$$

at epoch 30.

The complete training history was used to plot the training and validation loss.

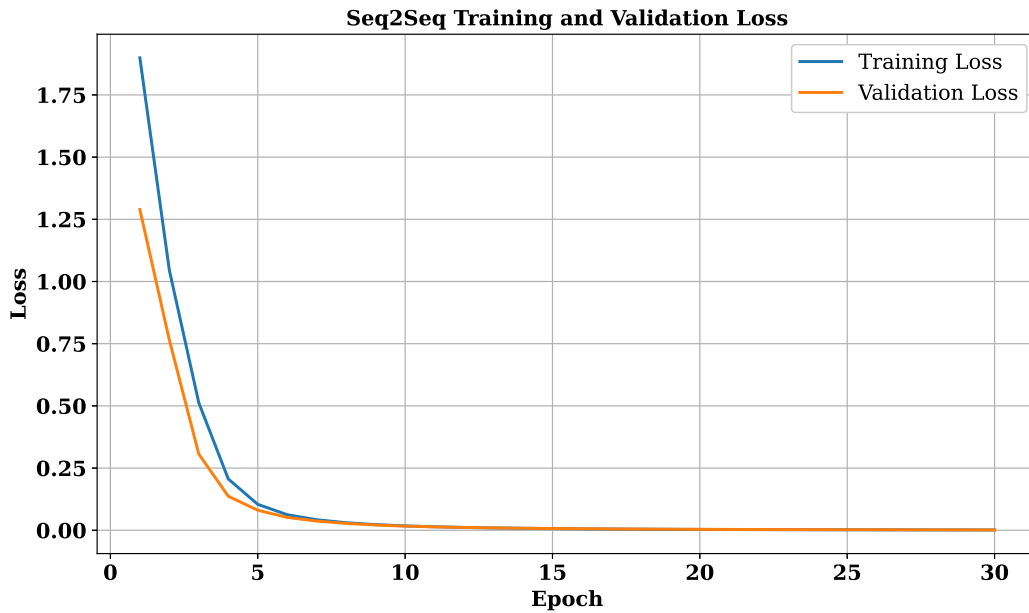


Figure 28: Training and validation loss of the Seq2Seq reversal model.

The training process required approximately:

40.35 seconds.

The training-loss value is not reported separately in the result table because the exported CSV did not contain a valid numerical value for that field. The report therefore uses only the recorded validation loss and the directly measured test results.

Seq2Seq Evaluation

The trained Seq2Seq model was evaluated on the 750 held-out test sequences.

Two levels of accuracy were considered:

1. Token-level accuracy: measures the proportion of correctly predicted output tokens.
2. Sequence-level accuracy: measures whether the complete predicted sequence exactly matches the target sequence.

The test results were:

Metric	Result
Test Token Accuracy	100.00%
Test Sequence Accuracy	100.00%
Correct Complete Sequences	750/750
Parameters	51,212
Training Time	40.35 s
Best Epoch	30
Best Validation Loss	0.001453

All 750 test sequences were generated exactly correctly. Therefore:

$$\frac{750}{750} \times 100 = 100.00\%$$

sequence-level accuracy was obtained.

The token-level and sequence-level accuracies were both 100%, indicating that every output token was correct and that all complete four-token reversed sequences matched their targets.

Example Seq2Seq Predictions

Representative input-output pairs from the test set are shown below.

No.	Input	Predicted Output
1	[4,2,3,7]	[7,3,2,4]
2	[7,1,8,7]	[7,8,1,7]
3	[1,8,2,8]	[8,2,8,1]
4	[3,7,6,5]	[5,6,7,3]
5	[2,2,1,9]	[9,1,2,2]

The examples show that the decoder successfully learned the reversal relationship. For example:

$$[4, 2, 3, 7] \rightarrow [7, 3, 2, 4]$$

and

$$[2, 2, 1, 9] \rightarrow [9, 1, 2, 2].$$

Since repeated tokens are allowed, the model also had to preserve the correct token order when the same value occurred more than once.

Inference

The Seq2Seq experiment showed that an encoder–decoder model can learn a fixed transformation from an input sequence to an output sequence.

In this experiment, the task was sequence reversal. The encoder converted the input sequence into a recurrent representation, and the decoder used this representation to generate the reversed sequence.

The model achieved 100.00% token accuracy and 100.00% exact sequence accuracy on all 750 test samples. Since the task was synthetic and deterministic, this shows that the model successfully learned the reversal mapping used in the experiment.

This result should be considered in the context of the controlled synthetic task and should not be taken as evidence that the same accuracy would be achieved for other sequence-to-sequence problems.

Additional Exercise 7: Different Input and Output Sequence Lengths

The Seq2Seq experiment was extended by using different input and output sequence lengths. The input sequence contained four tokens, while the target sequence contained five tokens.

The additional target token was generated as part of the synthetic sequence transformation task.

The experimental configuration was:

Parameter	Value
Input sequence length	4
Output sequence length	5
Training samples	3500
Validation samples	750
Test samples	750
Vocabulary size	12
Embedding dimension	32
LSTM latent dimension	64
Trainable parameters	51,212
Training time	40.11 s
Best epoch	30
Best validation loss	0.208676

The model obtained:

$$85.52\% \text{ token accuracy}$$

and

$$30.53\% \text{ sequence accuracy}.$$

Out of the 750 test sequences, 229 complete sequences were predicted exactly:

$$\frac{229}{750} \times 100 = 30.53\%.$$

The difference between token-level and sequence-level accuracy is important in this experiment. A sequence is counted as correct only when all five output tokens are correct. Therefore, even one incorrect token makes the complete sequence incorrect.

AE7 Example Predictions

Five representative test examples are shown below.

No.	Input	Target	Prediction
1	[2,9,2,3]	[3,2,9,2,7]	[3,2,9,2,7]
2	[4,3,1,1]	[1,1,3,4,9]	[1,1,3,4,9]
3	[5,1,3,1]	[1,3,1,5,1]	[1,3,1,5,1]
4	[1,4,7,8]	[8,7,4,1,2]	[8,7,4,1,7]
5	[6,3,8,6]	[6,8,3,6,5]	[6,8,3,6,9]

The first three examples were predicted exactly. In the fourth example, four of the five output tokens were correct, but the final token was predicted as 7 instead of 2. Similarly, in the fifth example, the final token was predicted as 9 instead of 5.

These examples show why token accuracy can remain high while exact sequence accuracy is lower. A single incorrect token makes the entire sequence incorrect.

AE7 Training Behaviour

The training and validation loss for the different-length Seq2Seq experiment are shown below.

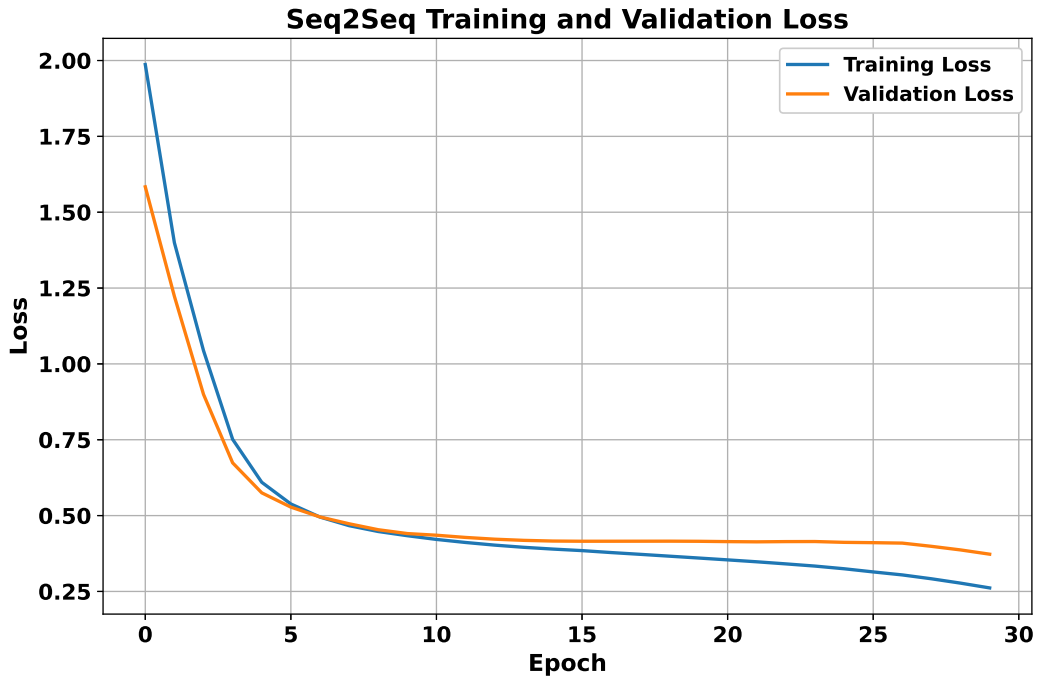


Figure 29: Training and validation loss for the Seq2Seq model with different input and output sequence lengths.

The best recorded validation loss was 0.208676 at epoch 30. The higher loss compared with the original four-to-four reversal task reflects the more difficult output-generation requirement in this additional experiment.

Seq2Seq Summary

The two Seq2Seq experiments demonstrated the effect of task complexity on sequence generation.

Metric	Original Reversal	Different Lengths
Input Length	4	4
Output Length	4	5
Test Samples	750	750
Token Accuracy	100.00%	85.52%
Sequence Accuracy	100.00%	30.53%
Exact Sequences	750/750	229/750
Parameters	51,212	51,212
Training Time	40.35 s	40.11 s

The original reversal task was solved correctly for all test sequences. When the task was changed to generate five output tokens from four input tokens, the token accuracy decreased to 85.52% and the exact sequence accuracy decreased to 30.53%.

These results show the importance of evaluating sequence generation at both the token level and the complete-sequence level. Complete-sequence accuracy is stricter because every generated token must be correct for the sequence to be counted as an exact match.

18. Consolidated Results

The complete experiment involved three main sequence-learning tasks:

1. Human activity recognition using Vanilla RNN, LSTM and GRU.
2. Video action classification using CNN-LSTM and CNN-GRU.
3. Sequence-to-sequence learning using an encoder-decoder LSTM.

The main HAR results obtained on the official UCI HAR test set are summarized below.

Metric	RNN	LSTM	GRU
Test Accuracy	65.22%	87.00%	89.01%
Macro Precision	68.03%	87.11%	89.38%
Macro Recall	63.86%	86.68%	88.97%
Macro F1-score	63.19%	86.78%	88.96%
Parameters	1,974	6,006	4,758
Training Time	28.97 s	20.70 s	19.89 s

The video classification results were:

Metric	CNN-LSTM	CNN-GRU
Test Accuracy	96.43%	89.29%
Macro Precision	96.67%	89.56%
Macro Recall	96.97%	90.91%
Macro F1-score	96.19%	88.85%
Parameters	168,643	126,723
Training Time	10.30 s	10.30 s

The Seq2Seq results were:

Metric	Original Seq2Seq	AE7
Input Length	4	4
Output Length	4	5
Test Samples	750	750
Token Accuracy	100.00%	85.52%
Sequence Accuracy	100.00%	30.53%
Exact Sequences	750/750	229/750
Parameters	51,212	51,212
Training Time	40.35 s	40.11 s

Overall, the experiments demonstrated three different uses of recurrent sequence modelling. Sensor data was directly classified as a temporal sequence, video frames were first converted into CNN features and then processed over time, and the Seq2Seq model generated an output sequence token by token.

19. Required Inferences

Inference 1: Effect of Recurrent Architecture

The three HAR models showed different test performance. The Vanilla RNN achieved 65.22% accuracy and 63.19% macro F1-score, while the LSTM achieved 87.00% accuracy and 86.78% macro F1-score. The GRU achieved 89.01% accuracy and 88.96% macro F1-score.

In this experiment, the gated recurrent models showed substantially higher measured test performance than the basic RNN.

Inference 2: Parameter Count

The Vanilla RNN contained 1,974 parameters, the LSTM contained 6,006 parameters, and the GRU contained 4,758 parameters.

The higher parameter count of the LSTM is due to its multiple gates and separate cell-state mechanism. The GRU used fewer parameters than the LSTM while still using gated information flow.

Inference 3: Training Time

The recorded training times were 28.97 seconds for the RNN, 20.70 seconds for the LSTM and 19.89 seconds for the GRU.

The training time did not directly follow the parameter count. The RNN had the fewest parameters but took longer in the recorded run. Therefore, the training time depends on the actual execution environment and implementation, not only on the number of parameters.

Inference 4: Sequence Length

The sequence-length experiment showed that the effect of temporal length depended on the model.

The RNN obtained its highest measured F1-score at 64 time steps, the LSTM at 128 time steps, and the GRU at 64 time steps.

Therefore, increasing the sequence length did not automatically improve the performance of every recurrent architecture.

Inference 5: Video Understanding

The video experiment used a two-stage approach. MobileNetV2 extracted spatial features from individual frames, and a recurrent network then processed the resulting feature sequence.

The CNN-LSTM achieved 96.43% test accuracy and 96.19% macro F1-score, while the CNN-GRU achieved 89.29% test accuracy and 88.85% macro F1-score.

In this particular three-class video experiment, the CNN-LSTM showed higher measured performance, while the CNN-GRU used fewer parameters.

Inference 6: Sequence-to-Sequence Learning

The original Seq2Seq model achieved 100% token accuracy and 100% exact sequence accuracy on all 750 test sequences.

The modified task with four input tokens and five output tokens achieved 85.52% token accuracy but only 30.53% exact sequence accuracy.

This shows the difference between token-level and sequence-level evaluation. A single incorrect token makes the complete sequence incorrect.

Inference 7: Overall Observation

The experiment demonstrated that recurrent networks can be used for different types of sequential information. Sensor measurements, CNN-extracted video features and symbolic token sequences all require the information to be processed in temporal order.

The experiments also showed that the model architecture and task definition affect the measured performance. Therefore, accuracy values should be interpreted along with the dataset, sequence length, model architecture and evaluation method used to obtain them.

20. Discussion Questions

1. Why are recurrent neural networks suitable for sequential data?

RNNs maintain a hidden state that is updated at each time step. This allows the current representation to contain information from previous time steps. Therefore, RNNs are suitable for data where the order of observations is important.

2. What is the role of the hidden state?

The hidden state acts as a running representation of the sequence. At each time step, it is updated using the current input and the previous hidden state.

3. Why is parameter sharing used in RNNs?

The same recurrent parameters are used at every time step. This reduces the number of parameters and allows the same temporal transformation to be applied throughout the sequence.

4. What is BPTT?

Backpropagation Through Time (BPTT) is the training procedure used for recurrent networks. The recurrent network is unfolded over the sequence, and the error is propagated backward through the time steps.

5. What causes the vanishing-gradient problem?

During BPTT, gradients are repeatedly multiplied through the recurrent connections. If these factors have magnitudes below one, the gradient can become very small after many time steps.

6. What causes the exploding-gradient problem?

If the repeated gradient multiplications produce values greater than one, the gradient can grow rapidly. This can make the training process unstable.

7. How does LSTM address the limitations of a Vanilla RNN?

LSTM introduces a cell state and gates to control the flow of information. The forget, input and output gates determine which information is retained, added and exposed.

8. What is the purpose of the forget gate?

The forget gate controls how much information from the previous cell state is retained.

9. What is the purpose of the input gate?

The input gate controls how much new candidate information is added to the cell state.

10. What is the purpose of the output gate?

The output gate controls how much information from the cell state is used to form the new hidden state.

11. How does GRU differ from LSTM?

GRU uses an update gate and a reset gate and does not maintain a separate cell state. LSTM uses a cell state together with forget, input and output gates.

12. Why does the GRU have fewer parameters than the LSTM in this experiment?

The GRU uses two gates and does not have a separate cell state, whereas the LSTM uses three gates and a separate cell-state mechanism.

13. Why is dropout used in the recurrent classifiers?

Dropout is used as a regularization technique to reduce excessive dependence on individual neurons and help control overfitting.

14. Why is softmax used in the final classification layer?

Softmax converts the final logits into a probability distribution over the six activity classes.

15. Why are precision, recall and F1-score reported along with accuracy?

Accuracy gives the overall proportion of correct predictions, while precision, recall and F1-score provide additional information about class-wise prediction performance.

16. Why is macro F1 useful for this experiment?

Macro F1 calculates the F1-score for each class and then takes their average. Therefore, each activity class contributes equally to the reported metric.

17. Why can SITTING and STANDING be confused?

Both activities can produce relatively low body-motion signals compared with dynamic activities. Their sensor patterns can therefore overlap in some sequences.

18. Why are WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS difficult to separate?

All three activities involve walking-related movement, so their acceleration and gyroscope patterns can have similar characteristics.

19. Why is temporal order important in HAR?

The same sensor values can have different meanings depending on how they change over time. Recurrent models use the order of observations to learn these temporal patterns.

20. Why is a CNN used before an RNN for video understanding?

The CNN extracts spatial information from individual frames, while the recurrent network processes the temporal sequence of the extracted features.

21. Why were CNN features represented as a sequence?

Each video contains multiple frames occurring in a particular order. Representing the frame-level CNN features as a sequence preserves this temporal order for the recurrent model.

22. What is the role of the encoder in Seq2Seq learning?

The encoder processes the input sequence and produces a representation that is used by the decoder to generate the output sequence.

23. What is the role of the decoder?

The decoder generates the output sequence step by step using the information obtained from the encoder and its previous decoding state.

24. Why is sequence-level accuracy lower than token accuracy in AE7?

Sequence-level accuracy requires every output token to be correct. Therefore, even one incorrect token causes the complete sequence to be counted as incorrect.

25. What is the main observation from the complete experiment?

The experiment showed that sequence models can process different types of sequential information, including sensor signals, video features and symbolic sequences. The measured performance depended on the recurrent architecture, sequence length and task complexity.

21. Additional Exercises

Additional Exercise 1: Effect of Number of Recurrent Units

The number of recurrent units was varied between 16 and 64 for RNN, LSTM and GRU models. The measured test performance did not show a consistent improvement when the number of units was increased.

Model	Units	Accuracy	Macro F1
RNN	16	35.87%	29.33%
LSTM	16	46.93%	41.17%
GRU	16	50.83%	48.38%
RNN	64	43.30%	36.33%
LSTM	64	43.64%	31.89%
GRU	64	51.82%	43.84%

These results show that increasing the number of recurrent units did not automatically improve the measured official-test performance in these additional runs.

Additional Exercise 2: LSTM vs GRU with 32 Units

The controlled 32-unit comparison was:

Metric	LSTM	GRU	Difference
Accuracy	87.00%	89.01%	2.01 pp
Macro F1	86.78%	88.96%	2.18 pp
Parameters	6,006	4,758	1,248 fewer
Training Time	20.70 s	19.89 s	0.81 s

In this experiment, the GRU used fewer parameters and obtained higher measured test accuracy and macro F1-score than the LSTM.

Additional Exercise 3: Two Recurrent Layers

Two recurrent layers with 32 units each were tested.

Model	Accuracy	Macro F1	Parameters
2-layer RNN	33.05%	25.89%	4,054
2-layer LSTM	46.18%	35.84%	14,326
2-layer GRU	55.62%	46.19%	11,094

The deeper recurrent models did not improve the official-test performance relative to the corresponding one-layer baseline in this experiment.

Additional Exercise 4: Bidirectional LSTM

A bidirectional LSTM with 32 units was compared with the unidirectional LSTM.

Model	Accuracy	Macro F1	Parameters
LSTM	87.00%	86.78%	6,006
BiLSTM	42.25%	31.28%	11,894

The BiLSTM had more parameters but showed substantially lower official-test performance in this particular run. Therefore, this result is treated as an observation from the given experimental setup rather than a general conclusion about bidirectional LSTMs.

Additional Exercise 5: Sequence Length and Computational Cost

The measured training times for different sequence lengths were:

Sequence Length	RNN	LSTM	GRU
32	23.46 s	11.33 s	15.35 s
64	17.58 s	16.39 s	16.62 s
128	23.74 s	19.46 s	18.06 s

The recorded training times were affected by execution and batching overhead, so they were not perfectly monotonic. However, longer sequences still required the recurrent model to process more time steps.

Additional Exercise 6: CNN–LSTM vs CNN–GRU

The two video recurrent models were compared using the same CNN feature extractor.

Metric	CNN–LSTM	CNN–GRU
Accuracy	96.43%	89.29%
Macro F1	96.19%	88.85%
Parameters	168,643	126,723
Training Time	10.30 s	10.30 s

CNN–LSTM obtained higher measured test performance in this three-class experiment, while CNN–GRU used fewer parameters.

Additional Exercise 7: Different Input and Output Lengths

The input sequence contained four tokens while the output contained five tokens. The model obtained:

85.52% token accuracy

and

30.53% sequence accuracy

.

A total of 229 out of 750 test sequences were predicted completely correctly.

The difference between the two metrics demonstrates the stricter nature of complete-sequence evaluation.

22. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.

3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>